Financial SDEs: GBM Benchmark and Non-Affine Volatility

Coupled convergence, reference uncertainty, and a moment-balanced factor scheme

MTH321 – Project 1: Numerical Solution of Initial Value Problems
Team D

Xihan Chen / 2360253 Bowen Duan / 2362211
Shuyu Teng / 2362204 Conghao Liang / 2362212

7 October 2026

Abstract

An exact stochastic benchmark is useful for validating an algorithm but can conceal whether discretisation is necessary. We first compare exact geometric Brownian motion (GBM), price-space Euler–Maruyama (EM), and scalar Milstein on common Brownian paths. With 65,536 independent paths, measured terminal mean-absolute-error orders are 0.4975 and 0.9972. An exact formula for the discrete expectation separates weak time-step bias from Monte Carlo uncertainty. We then simulate the prescribed non-affine stochastic-volatility model using log-price EM and nested reference grids. Its empirical strong rate is 0.5003 relative to the finest reference, while most estimated call-payoff biases are statistically unresolved. A model-specific extension matches the factor's exact mean dynamics and stationary variance and removes EM's second-moment step-size restriction. A coarse-step stress test exposes both EM moment instability and a misleading Monte Carlo confidence interval caused by rare large values. At a common GBM error ceiling, Milstein uses sixteen times fewer path updates than EM; exact terminal simulation remains preferable for GBM. All numerical claims are tied to executable source, recorded outputs and explicit limitations.

Keywords: stochastic differential equations; Euler–Maruyama; Milstein; Brownian coupling; Monte Carlo uncertainty; stochastic volatility.

Submission package. Team D supplies this report, a portable code ZIP with README and recorded outputs, and the approved 17-page presentation PDF. The report follows the supplied background–model–methods–stability–experiments–conclusions structure, specialised to the Financial SDE direction. Core numerical figures use inline data; supplementary figures and source listings are included in the companion source bundle.

Team records. The intellectual-ownership declaration in Appendix A and one labelled individual contribution statement per member in Appendix B are included, with member names, student IDs and signatures.

Contents

1 Background, question and assessment scope **3**

2 Models, assumptions and exact identities **3**
2.1 Probability space, units and parameters . . . 3
2.2 GBM: Itô transformation and probability distribution . . . 4
2.3 Prescribed non-affine stochastic-volatility model . . . 5
2.4 Factor moments as an additional analytic benchmark . . . 5

3 Numerical methods and convergence concepts **6**
3.1 Exact simulation and price-space Euler–Maruyama . . . 6
3.2 Derivation of the scalar Milstein correction . . . 6
3.3 An exact weak-error formula for the tested observable . . . 7
3.4 Log-price EM and correlated increments . . . 7
3.5 Why scalar Milstein does not transfer to the joint model . . . 7

4 Stability, positivity and a derived extension **8**
4.1 Mean-square growth of GBM . . . 8
4.2 When price-space methods become negative . . . 8
4.3 Exact discrete factor second-moment stability . . . 8
4.4 Moment-balanced factor scheme . . . 9

5 Experimental design, uncertainty and reproducibility **9**
5.1 Nested Brownian construction . . . 9
5.2 Error definitions and statistical intervals . . . 10
5.3 Reference resolution and falsification rules . . . 10
5.4 Control variate and separation of error sources . . . 11
5.5 Implementation choices, unit checks and environment . . . 11

6 Results and interpretation **12**
6.1 GBM: strong and weak convergence . . . 12
6.2 Distribution, quantiles and analytic moments . . . 13
6.3 Non-affine reference validation and strong convergence . . . 14
6.4 Weak-payoff uncertainty and a more precise expected payoff . . . 15
6.5 Positivity counterexample . . . 16
6.6 Extension: preserved moments, limited price advantage . . . 17
6.7 Measured cost at a common accuracy target . . . 18
6.8 Transient visualisation . . . 19
6.9 Guide-compliant rate overlays and cost comparison . . . 19

7 Limitations, conclusions and practical recommendations **20**

A Part A – Team AI Transparency Log **23**

B Part B – Individual Contribution Statements (ICS) **24**

C Reproducibility and submission instructions **29**

D Presentation and viva preparation **30**

E Complete executable computational source **31**

1 Background, question and assessment scope

Before choosing a numerical method for an asset-price model, we need to ask whether time discretisation is necessary. Geometric Brownian motion (GBM) has an explicit solution, so we can compare a numerical approximation with the exact value along the same Brownian path. This makes it a useful starting point for testing our implementation. The non-affine stochastic-volatility model is more challenging because we do not have a practical exact sampler. We therefore use fine-grid numerical solutions as references and examine how their remaining errors affect our conclusions.

This project asks two questions. First, how reliable are our convergence results when we replace an exact solution with a numerical reference? Second, can an update designed to preserve selected moments of the volatility factor improve stability and computational performance? We address these questions using direction (4) of the Problem Pack [2]. The SDE-specific requirements take precedence over the general brief's requirements for deterministic Euler, RK4, Newton and adaptive ODE methods [1]. Brownian paths cannot be treated as ordinary differentiable inputs to an ODE solver. Our analysis therefore focuses on strong and weak errors, second-moment behaviour, positivity and Monte Carlo uncertainty.

We extend the baseline study by deriving a recurrence for the factor's discrete second moment, identifying the EM stability threshold and constructing a moment-balanced update. We then compare this update with EM using shared Brownian paths, including a long-horizon experiment with large time steps. The construction follows established exponential and moment-fitting ideas; we do not present it as a new numerical method. Instead, we assess what it preserves and whether those properties lead to more accurate prices or lower computational cost. We also use a stock-price control variate to reduce sampling uncertainty and examine a weak-convergence claim that the available results do not support. These experiments help distinguish theoretical properties from conclusions that can be justified by the simulations.

Table 1: Evidence map to the highest-band descriptors in the supplied rubric [3]. This is a coverage map, not a predicted mark.

Dimension	Concrete evidence and remaining boundary
A: mathematics	Itô derivation; exact moments; EM/Milstein orders; model-specific stability and positivity; numerical agreement and deviations.
B: depth/novelty	Derived moment-balanced update, proof of invariant second moment, stress counterexample and coupled comparison. No research-priority claim.
C: implementation	Direct vectorised solvers, fixed random streams, 12 executable checks, README/requirements/script export. Historical iterative Git evidence must come from actual team work.
D: experiments	Coupled errors with intervals, reference refinement, matched-error timings, failed weak-rate/stress checks, embedded transient animation.
E: communication	Background-to-conclusion structure, pseudocode, limitations and three substantive AI claim audits. Presentation rehearsal remains team work.
F: individuals	Authentic 200-word ICS records, weekly tickets/challenges, viva and peer reviews cannot be generated as evidence. Appendix B gives the exact completion fields.

2 Models, assumptions and exact identities

2.1 Probability space, units and parameters

In the following study, all our simulations use independent paths, while simulations of the same path at different step sizes are intentionally kept correlated. All simulations are performed on a filtered probability space carrying Brownian motion. The state processes are adapted, and the

integrals are Itô integrals. Time is measured in years. Drift, mean reversion and diffusion use the corresponding annualised units.

The expression $\mathbb{E}[(S_T - 100)^+]$ below represents *undiscounted expected payoffs* and does not automatically correspond to arbitrage-free option prices. If a risk-neutral interpretation is additionally adopted, with $r = \mu = 0.05$ and no dividends, the call option value in the non-affine model is $e^{-rT}\mathbb{E}[(S_T - 100)^+]$. This interpretation is an additional assumption; this report does not claim market calibration or estimation of the volatility risk premium, and it does not provide investment advice.

Table 2: We use the parameters specified in the Problem Pack for the non-affine model. The GBM parameters are chosen to make the mean bias sufficiently visible and are not calibrated to market data.

Parameter	GBM	Non-affine model
S_0, T	100, 1	100, 1
μ	0.20	0.05
σ	0.30	$g(Y)$
Y_0, θ, κ	not applicable	0, -0.2, 2
ξ, ρ	not applicable	0.6, -0.7
$\sigma_{\min}$, $\sigma_{\max}$	not applicable	0.10, 0.50

2.2 GBM: Itô transformation and probability distribution

For $dS_t = \mu S_t\,dt + \sigma S_t\,dW_t$, with $S_0 > 0$, apply Itô's formula to $f(s) = \log s$:

$$
\begin{aligned}
d\log S_t &= f'(S_t)dS_t + \tfrac{1}{2}f''(S_t)(dS_t)^2 && (1)\\
&= \frac{1}{S_t}(\mu S_t dt + \sigma S_t dW_t) - \frac{\sigma^2 S_t^2}{2S_t^2}dt && (2)\\
&= (\mu - \tfrac{1}{2}\sigma^2)dt + \sigma dW_t. && (3)
\end{aligned}
$$

Therefore:

$$S_t = S_0 \exp\{(\mu - \sigma^2/2)t + \sigma W_t\} > 0. \tag{4}$$

For $Z \sim N(0,1)$, completing the square in the Gaussian integral gives $\mathbb{E}[e^{aZ}] = e^{a^2/2}$. Therefore, for any real number p:

$$\mathbb{E}[S_T^p] = S_0^p \exp\{p\mu T + \tfrac{1}{2}p(p-1)\sigma^2 T\}. \tag{5}$$

In particular,

$\mathbb{E}[S_T] = S_0 e^{\mu T}, \qquad \mathrm{Var}(S_T) = S_0^2 e^{2\mu T}(e^{\sigma^2 T} - 1).$

Its probability density function and quantile function are:

$$f_T(s) = \frac{1}{s\sigma\sqrt{2\pi T}} \exp\left[-\frac{(\log(s/S_0) - (\mu - \sigma^2/2)T)^2}{2\sigma^2 T}\right], \quad s > 0, \tag{6}$$

$$Q(p) = S_0 \exp\{(\mu - \sigma^2/2)T + \sigma\sqrt{T}\,\Phi^{-1}(p)\}. \tag{7}$$

The results above provide independent distributional checks and an exact pathwise benchmark. They also show that, in practical applications where only terminal prices are needed, we do not need to simulate GBM using methods with discretisation bias.

2.3 Prescribed non-affine stochastic-volatility model

The second benchmark model is:

$$dX_t = [\mu - \tfrac{1}{2}g(Y_t)^2]dt + g(Y_t)dW_t^{(1)}, \qquad S_t = e^{X_t}, \tag{8}$$

$$dY_t = \kappa(\theta - Y_t)dt + q(Y_t)dW_t^{(2)}, \qquad q(y) = \xi\sqrt{1+y^2}, \tag{9}$$

$$g(y) = \sigma_{\min} + \frac{\sigma_{\max} - \sigma_{\min}}{1+e^{-y}}, \qquad d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\, dt. \tag{10}$$

The logistic function is evaluated using `scipy.special.expit` to avoid numerical overflow when directly evaluating the exponential function. We have $0.1 < g(y) < 0.5$, $|g'(y)| \le 0.1$ and $|q'(y)| \le 0.6$. The drift and diffusion coefficients for the state variables (X, Y) satisfy a global Lipschitz condition and have linear growth. Therefore, on a finite time interval, the model has a unique, non-explosive strong solution.

The diffusion coefficient is smooth at $Y = 0$; $\sqrt{1+Y^2}$ is not $\sqrt{Y}$, so it does not require the volatility factor Y to be nonnegative.

For a smooth test function $u(x, y)$, its generator is:

$$\mathcal{L}u = (\mu - \tfrac{1}{2}g^2)u_x + \kappa(\theta - y)u_y + \tfrac{1}{2}g^2 u_{xx} + \rho g q\, u_{xy} + \tfrac{1}{2}q^2 u_{yy}. \tag{11}$$

The logistic coefficients and nonlinear factor variance are not affine in the state. We treat the fine-grid results as numerical reference solutions rather than exact solutions because we do not have a general exact transition formula or a practically usable exact sampling method for the model above.

Applying Itô's formula to e^X cancels the drift correction term $-g^2/2$:

$$dS_t = \mu S_t dt + g(Y_t)S_t dW_t^{(1)}. \tag{12}$$

Since g is bounded, Novikov's condition holds on any finite time interval. Therefore, the discounted stochastic exponential is a true martingale, and:

$$\mathbb{E}[S_T] = S_0 e^{\mu T}. \tag{13}$$

This moment identity remains valid even when the two noises driving the model are correlated. Bounded volatility also ensures that positive-order moments of the stock price are finite; however, simulated paths remaining positive does not prove numerical accuracy.

2.4 Factor moments as an additional analytic benchmark

Let $m_1(t) = \mathbb{E}[Y_t]$ and $m_2(t) = \mathbb{E}[Y_t^2]$. Taking expectations in the factor SDE and in the equation obtained by applying Itô's formula to Y^2 gives:

$$m_1' = \kappa(\theta - m_1), \qquad m_1(t) = \theta + (Y_0 - \theta)e^{-\kappa t}, \tag{14}$$

$$m_2' = -(2\kappa - \xi^2)m_2 + 2\kappa\theta m_1 + \xi^2. \tag{15}$$

Let $a = 2\kappa - \xi^2$. When $a \neq \kappa$ and $a \neq 0$:

$$m_2(t) = e^{-at}Y_0^2 + \frac{2\kappa\theta^2 + \xi^2}{a}(1 - e^{-at}) + \frac{2\kappa\theta(Y_0 - \theta)}{a - \kappa}(e^{-\kappa t} - e^{-at}). \tag{16}$$

When the decay rates coincide, the corresponding result can be obtained by taking a continuous limit. Under the benchmark parameters of this project, $a = 3.64 > 0$, and the stationary central second moment is:

$$v_\infty = \frac{\xi^2(1+\theta^2)}{2\kappa - \xi^2} = 0.102857142857. \tag{17}$$

This result describes the moments of the factor, rather than a closed-form transition formula for the joint price-factor process. It provides a check independent of the fine-grid price reference solution.

3 Numerical methods and convergence concepts

3.1 Exact simulation and price-space Euler–Maruyama

For $h = T/N$ and independent $\Delta W_n \sim N(0, h)$, the exact GBM update is

$$S^{\text{ex}}_{n+1} = S^{\text{ex}}_n \exp\{(\mu - \sigma^2/2)h + \sigma\Delta W_n\}. \tag{18}$$

There is no time-discretisation error at grid points and EM applied to $X = \log S$ gives this same transition because of the constant transformed drift and diffusion. Thus, it would be wrong to use log-GBM to measure the strong order of scalar Milstein.

Then, integrating the price SDE over $[t_n, t_{n+1}]$ and freezing both coefficients at S_n gives

$$S^{\text{EM}}_{n+1} = S^{\text{EM}}_n(1 + \mu h + \sigma\Delta W_n). \tag{19}$$

The notebook implements this directly; NumPy's product computes the product of the scalar factors with no Python loop over individual paths.

3.2 Derivation of the scalar Milstein correction

For $dS = a(S)dt + b(S)dW$, we substitute the leading stochastic increment $S_s - S_n \simeq b(S_n)(W_s - W_n)$ into $b(S_s)$. Its first derivative generates

$$b'(S_n)b(S_n)\int_{t_n}^{t_{n+1}} (W_s - W_{t_n})\, dW_s = \tfrac{1}{2}b'b(S_n)[(\Delta W_n)^2 - h], \tag{20}$$

since Itô's formula applied to $(W_s - W_{t_n})^2$ gives the integral exactly. With $b(s) = \sigma s$,

$$S^{\text{Mil}}_{n+1} = S^{\text{Mil}}_n\{1 + \mu h + \sigma\Delta W_n + \tfrac{1}{2}\sigma^2[(\Delta W_n)^2 - h]\}. \tag{21}$$

The correction has zero conditional mean and captures the leading quadratic variation fluctuation. Smooth scalar coefficients with suitable moment bounds give strong order one and standard EM has strong order one-half. Moreover, both of them have weak order one for sufficiently smooth test functions. These standard order concepts and simulation methods are described by Higham [4] and the problem-specific formulas and tests here are derived and implemented explicitly.

For GBM, expand the exact factor with $\Delta W = O_{L^p}(h^{1/2})$:

$$e^{(\mu-\sigma^2/2)h+\sigma\Delta W} = 1 + \mu h + \sigma\Delta W + \tfrac{1}{2}\sigma^2[(\Delta W)^2 - h] + O_{L^2}(h^{3/2}). \tag{22}$$

EM drops a centred local term of size $O_{L^2}(h)$. The accumulation of its stochastic fluctuations over $O(h^{-1})$ steps explains the $O(h^{1/2})$ global scale. Milstein retains this term. With the required local-mean bounds and finite moments, the remaining fluctuations yield $O(h)$ global error. But this scaling explanation is not a replacement for the assumptions in a convergence theorem.

3.3 An exact weak-error formula for the tested observable

Conditioning on S_n in either price scheme gives $\mathbb{E}[S_{n+1} \mid S_n] = (1+\mu h)S_n$. Induction yields

$$\mathbb{E}[S_N^{\mathrm{EM}}] = \mathbb{E}[S_N^{\mathrm{Mil}}] = S_0(1+\mu h)^{T/h}. \tag{23}$$

For $1+\mu h > 0$, expand $(T/h)\log(1+\mu h) = \mu T - \mu^2 T h/2 + O(h^2)$. Therefore

$$\mathbb{E}[S_N] - \mathbb{E}[S_T] = -\tfrac{1}{2} S_0 e^{\mu T} \mu^2 T h + O(h^2). \tag{24}$$

The bias constant is nonzero for the chosen $\mu = 0.2$ and the weak mean order is exactly one asymptotically. No weak-order experiment based only on the first moment could distinguish methods with $\mu = 0$, since this observable has zero bias at every step size. Thus, equal expected means do not imply equal sampled means or equal distributions.

3.4 Log-price EM and correlated increments

Generate independent $Z_{1,n}, Z_{2,n} \sim N(0,1)$ and set

$$\Delta W_n^{(1)} = \sqrt{h} Z_{1,n}, \qquad \Delta W_n^{(2)} = \sqrt{h}(\rho Z_{1,n} + \sqrt{1-\rho^2} Z_{2,n}). \tag{25}$$

Both increments have variance h and covariance ρh. The left-endpoint EM is

$$X_{n+1} = X_n + (\mu - \tfrac{1}{2} g(Y_n)^2)h + g(Y_n)\Delta W_n^{(1)}, \tag{26}$$

$$Y_{n+1} = Y_n + \kappa(\theta - Y_n)h + q(Y_n)\Delta W_n^{(2)}, \qquad S_{n+1} = e^{X_{n+1}}. \tag{27}$$

Both updates use the old Y_n. Using Y_{n+1} in the same price step would change both the scheme and its limiting drift.

Conditioning on the past, rather than on the future factor increment, the Gaussian exponential gives

$$\mathbb{E}[S_{n+1} \mid \mathcal{F}_{t_n}] = e^{\mu h} S_n. \tag{28}$$

Thus log EM has exactly the correct first price moment at every uniform grid size. This is an unusually useful oracle and highlights the warning that a correct mean can coexist with wrong paths and wrong option payoffs. Positivity holds mathematically under exponentiation and the code additionally rejects infinite states or $|X| \geq 700$ to avoid floating-point overflow.

3.5 Why scalar Milstein does not transfer to the joint model

In independent Brownian coordinates B_1, B_2, let $c = \sqrt{1-\rho^2}$. The diffusion vector fields are $b_1 = (g, \rho q)^T$ and $b_2 = (0, cq)^T$. Using the convention $[b_1, b_2] = Db_2 b_1 - Db_1 b_2$, their commutator is

$$[b_1, b_2] = (-cqg', 0)^T \neq 0. \tag{29}$$

Therefore, a full strong order-one multidimensional Milstein method generally needs cross iterated integrals including stochastic area information. Applying scalar corrections independently is not justified. We therefore keep EM as the prescribed reference method and do not call the extension below 'multidimensional Milstein'.

4 Stability, positivity and a derived extension

4.1 Mean-square growth of GBM

From (5), we have the exact second moment which is multiplied over one step by $e^{(2\mu+\sigma^2)h}$. The independence of each fresh increment from the current state gives

$$R_2^{\mathrm{EM}}(h) = (1+\mu h)^2 + \sigma^2 h, \tag{30}$$

$$R_2^{\mathrm{Mil}}(h) = (1+\mu h)^2 + \sigma^2 h + \tfrac{1}{2}\sigma^4 h^2. \tag{31}$$

The cross terms vanish because of $\mathbb{E}[Z] = \mathbb{E}[Z^3] = 0$ and $\mathbb{E}[(Z^2-1)^2] = 2$. Mean-square decay for the scalar linear test SDE requires $R_2 < 1$. Thus, EM needs $h < -(2\mu+\sigma^2)/\mu^2$ when $2\mu+\sigma^2 < 0$ and Milstein needs $h < -(2\mu+\sigma^2)/(\mu^2+\sigma^4/2)$ with degenerate cases treated directly.

For the GBM parameters $2\mu+\sigma^2 = 0.49 > 0$, the exact second moment grows. Requiring $R_2 < 1$ here would label correct physical growth an instability. We compare distributional moments and step-size errors instead and find that a higher strong order does not imply a larger mean-square stability region.

4.2 When price-space methods become negative

From a positive price, the EM factor is negative precisely when $Z < -(1+\mu h)/(\sigma\sqrt{h})$, so

$$p_{\mathrm{EM}}(h) = \Phi\left(-\frac{1+\mu h}{\sigma\sqrt{h}}\right) > 0 \tag{32}$$

for finite h and nonzero σ. For Milstein, we complete the square:

$$F_{\mathrm{Mil}}(Z) = \tfrac{1}{2}\sigma^2 h\left(Z + \frac{1}{\sigma\sqrt{h}}\right)^2 + \underbrace{\tfrac{1}{2} + (\mu - \tfrac{1}{2}\sigma^2)h}_{m(h)}. \tag{33}$$

A negative update is possible if and only if $m(h) < 0$. If $m(h) \geq 0$, a zero at a single Gaussian point has probability zero and the multiplier is positive almost surely. When $m(h) < 0$, define $r = \sqrt{-2m(h)/(\sigma^2 h)}$, $z_c = -1/(\sigma\sqrt{h})$, and

$$p_{\mathrm{Mil}}(h) = \Phi(z_c + r) - \Phi(z_c - r). \tag{34}$$

For the baseline $\mu = 0.2$, $\sigma = 0.3$, Milstein is positive almost surely for all $h > 0$. Its positivity depends on the parameters. The stress case $\mu = 0.05$, $\sigma = 1.5$ deliberately tests this distinction.

Negative prices do not automatically produce negative call payoffs after taking a positive part but they distort distributions, path-dependent cashflows and put payoffs. Furthermore, clipping S to zero changes the method and its weak bias, so it is not an innocuous repair. These experiments measure one-step sign failures rather than the probability of crossing zero over a multi-step path.

4.3 Exact discrete factor second-moment stability

Put $U_n = Y_n - \theta$. Standard EM gives

$$U_{n+1} = (1-\kappa h)U_n + \xi\sqrt{1+(U_n+\theta)^2}\,\Delta W_n^{(2)}. \tag{35}$$

If $\mathbb{E}[U_0] = 0$, then $\mathbb{E}[U_n] = 0$ for every n, and the exact recurrence is

$$v_{n+1} = A_h v_n + \xi^2 h(1+\theta^2), \quad A_h = (1-\kappa h)^2 + \xi^2 h. \tag{36}$$

Hence a finite attracting second moment requires

$$0 < h < \frac{2\kappa - \xi^2}{\kappa^2} = 0.91. \tag{37}$$

The EM stationary variance, when stable, is

$$v_{\infty,h}^{\mathrm{EM}} = \frac{\xi^2(1+\theta^2)}{2\kappa - \xi^2 - \kappa^2 h} > v_\infty. \tag{38}$$

For a nonzero initial mean displacement, a known transient forcing $2\theta\xi^2 h\mathbb{E}[U_n]$ appears. That does not change the attracting stationary value when the mean and second-moment recurrences are stable. Thus, (36) is exact for the nonlinear factor coefficient, unlike a frozen-Jacobian heuristic.

4.4 Moment-balanced factor scheme

We choose $a_h = e^{-\kappa h}$ and $c_h^2 = \xi^2(1 - e^{-2\kappa h})/(2\kappa)$ and keep the log-price update in (26). Next, we replace only the factor step by

$$Y_{n+1} = \theta + a_h(Y_n - \theta) + \frac{c_h}{\sqrt{h}}\sqrt{1 + Y_n^2}\,\Delta W_n^{(2)}. \tag{39}$$

The same Brownian increment preserves the intended within-step correlation and coarse/fine coupling. The coefficients tend to their EM counterparts as $h \downarrow 0$. This is not an exact Ornstein–Uhlenbeck path update, because the actual factor diffusion depends on Y. Rescaling an endpoint Brownian increment does not reproduce a stochastic convolution on that path.

Proposition 1. *If $2\kappa > \xi^2$, scheme (39) has the exact conditional factor mean and the exact attracting stationary variance (17) for every $h > 0$. The log-price first-moment identity (28) remains exact.*

Proof. The fresh centred increment gives $\mathbb{E}[Y_{n+1} \mid Y_n] = \theta + a_h(Y_n - \theta)$, which equals the continuous conditional mean. At centred mean, the central second moment follows

$$v_{n+1} = \widetilde{A}_h v_n + c_h^2(1+\theta^2), \quad \widetilde{A}_h = a_h^2 + c_h^2 = e^{-2\kappa h} + \frac{\xi^2}{2\kappa}(1 - e^{-2\kappa h}). \tag{40}$$

This is strictly between zero and one since $\xi^2/(2\kappa) < 1$. Solving the fixed-point equation cancels $1 - e^{-2\kappa h}$ and yields $v_\infty = \xi^2(1+\theta^2)/(2\kappa - \xi^2)$. The mean displacement decays as a_h^n. Thus, its extra second-moment forcing also decays. Finally, the price calculation conditions only on Y_n and is unchanged. □

Exact stationary variance does not imply an exact stationary distribution, transient variance, prices or improved strong order. These claims are tested separately below. The word 'Balanced' in the code is a descriptive flag for this explicit formula rather than an appeal to an unspecified solver library.

5 Experimental design, uncertainty and reproducibility

5.1 Nested Brownian construction

For each independent batch, Brownian increments are first generated on the finest grid, with step size $h_f = T/N_f$. Whenever $N_f = rN$, increments for the coarser step $h = T/N$ are obtained by summing consecutive groups of r fine-grid increments:

$$\Delta W_n^{(h)} = \sum_{j=0}^{r-1} \Delta W_{rn+j}^{(h_f)}. \tag{41}$$

The same procedure is applied separately to both correlated Brownian components. This construction preserves the required Gaussian distributions and correlation at every resolution, while ensuring that all approximations are driven by the same Brownian sample path. Generating independent noise for coarse and fine grids would prevent their pathwise differences from vanishing under refinement and would therefore invalidate a strong-convergence comparison.

Listing 1: Coupled experiment pseudocode; reference and method paths use the same increments.

```
For each independent batch:
    generate the finest independent Gaussian arrays
    construct correlated Brownian increments if needed
    for every grid N dividing N_f:
        sum consecutive fine increments into N blocks
        advance each method using the aggregated increments
    GBM: compare terminals with exact exp formula using total W_T
    Non-affine: compare terminals/payoffs with each fine EM reference
    retain pathwise differences and independent-batch means
Aggregate means, standard errors, and joint batch-bootstrap slopes
Repeat reference refinement; run analytic checks and failure cases
Time integration at the same predeclared accuracy ceiling
```

5.2 Error definitions and statistical intervals

Let R_i denote either the exact terminal value or a clearly identified numerical reference for path i. Define $A_i(h) = |S_i^{(h)} - R_i|, \qquad D_i(h) = \phi(S_i^{(h)}) - \phi(R_i)$. The estimated strong discrepancy and signed weak bias are given below. For a sample quantity V, the standard error and approximate 95% confidence interval are:

$$\widehat{e}_s = \frac{1}{M}\sum A_i, \quad \widehat{b}_\phi = \frac{1}{M}\sum D_i, \quad \mathrm{SE}(\overline{V}) = \frac{s_V}{\sqrt{M}}, \quad \mathrm{CI}_{95\%} = \overline{V} \pm 1.96\,\mathrm{SE}(\overline{V}). \tag{42}$$

where s_V is the sample standard deviation calculated with denominator $M-1$. These intervals quantify Monte Carlo uncertainty for individual estimates. They are neither simultaneous confidence bands nor bounds on the discretisation bias of the reference solution. Weak differences retain their signs: averaging absolute payoff differences would instead measure an L^1 payoff discrepancy.

Strong convergence rates are estimated by ordinary least squares using $\log \widehat{e}_s = \alpha + p \log h$. Uncertainty in the fitted slope is assessed with 500 bootstrap resamples of complete independent batches. Resampling whole batches preserves the dependence between estimates obtained on different grids. The GBM fit uses the seven finest grid levels. For the non-affine model, the fit uses $N = 8, 16, 32, 64, 128$, keeping the tested grids sufficiently separated from the reference resolution. These fitting ranges are selected on resolution grounds rather than to obtain a preferred convergence rate.

For the GBM weak-mean analysis, the negative of the signed bias estimate is fitted using $N \le 256$ for EM and all tested levels for Milstein. The finest EM levels are excluded because their sampling precision is insufficient for a reliable weak-rate estimate. Equation (23) supplies an exact, sampling-noise-free comparison at every level. For the non-affine model, a weak convergence rate is reported only when a consistent sequence of statistically resolved biases supports a power-law relationship. A few individual intervals that exclude zero are insufficient.

5.3 Reference resolution and falsification rules

The GBM experiments use 65,536 paths and grid sizes $N = 4, 8, \ldots, 1024$. The non-affine experiments use 32,768 paths, with coarse grids $N = 8, 16, \ldots, 256$ and reference grids $N_r = 1024, 2048, 4096$. Paths are processed in batches of 1,024.

Reference sensitivity is assessed using two operational criteria: a change of less than 5% in the strong discrepancy at the finest displayed test level, and a change of less than 0.02 in the fitted

convergence rate. These criteria support the robustness of the broad empirical-rate conclusion; they do not establish the accuracy of the error relative to the exact continuous-time solution. Differences between successive reference resolutions are reported separately, and their Monte Carlo confidence intervals are not interpreted as bounds on the continuum error.

The experimental design allows the proposed conclusions to fail. EM and Milstein may not exhibit their expected GBM rates; reference refinement may change the estimated non-affine rate; weak biases may remain indistinguishable from sampling noise; second-moment instability may occur even when simulated paths remain finite; and the moment-preserving extension may fail to reduce computational cost at matched accuracy. These possibilities are evaluated without changing model parameters in response to the observed results.

5.4 Control variate and separation of error sources

By Equation (28), the quantity $C = S_{N_r} - 100e^{0.05}$ has zero expectation, including under the discrete reference scheme. For the call payoff $P = (S_{N_r} - 100)^+$, the control-variate estimator is

$$\widehat{P}_{\mathrm{cv}} = \frac{1}{M_e} \sum_{i \in \text{evaluation}} (P_i - \widehat{\beta} C_i), \qquad \beta^* = \frac{\mathrm{Cov}(P, C)}{\mathrm{Var}(C)}. \tag{43}$$

The coefficient $\widehat{\beta}$ is estimated from a separate pilot sample of 4,096 paths. The remaining 28,672 paths form the evaluation sample. Conditional on the pilot sample, the estimator remains unbiased for the expected payoff of the discrete reference model. For a fixed coefficient β, the variance of the adjusted payoff is $\mathrm{Var}(P) - 2\beta\,\mathrm{Cov}(P, C) + \beta^2\,\mathrm{Var}(C)$, which is minimised at $\beta = \beta^*$. Raw and adjusted estimates are compared using the same evaluation paths. This procedure reduces sampling variance, but it does not correct the reference scheme's time-discretisation bias.

5.5 Implementation choices, unit checks and environment

The scalar methods and Itô–Taylor interpretation follow standard SDE numerical analysis [5, 4]. The implementation is organised around four functions: `gbm`, `sv`, `summary` and `slope_ci`, with source code provided in Appendix E. The numerical schemes and the modified factor update are implemented directly and vectorised across paths. NumPy [6] supplies array operations and random-number generation; SciPy [7] provides `expit`, `ndtr` and `ndtri`; and Matplotlib and Pillow generate the plots and animation. No external SDE solver or undisclosed market dataset is used. Each experiment uses a separate random stream constructed through `SeedSequence([20260916, experiment_id])`, and bootstrap resampling also uses a fixed seed.

The reported computations were performed with Python 3.12.14 and NumPy 2.5.3 on macOS/ARM64. Batch processing limits memory requirements by avoiding simultaneous storage of all fine-grid paths. Twelve executable checks examine exact-transition composition, zero-diffusion behaviour, the scalar Milstein correction, reduction of both factor schemes to exact GBM when the factor is frozen, Brownian variance and correlation, two analytic mean identities, two convergence-rate ranges, and the balanced scheme's variance identity. In the frozen-factor test, $\xi = 0$ and $Y_0 = \theta = 0$, giving $g(Y) \equiv 0.3$. This provides an independent exact benchmark rather than a comparison with another approximate solver.

Moment checks use an acceptance range of four standard errors. The relative variance tolerance and the correlation tolerance for Brownian increments are both 0.002, and the observed deviations are reported explicitly. The accepted empirical strong-rate ranges are $(0.45, 0.60)$ for EM and $(0.9, 1.1)$ for Milstein. These checks are intended to detect implementation problems, not to establish general convergence theorems.

Algorithm 1 One batch of coupled GBM and non-affine convergence measurements

Require: Finest grid N_f, nested coarse grids $N \mid N_f$, batch size B, seed
Ensure: Per-grid strong errors and signed weak differences on common paths
Draw independent Gaussian arrays Z_1, Z_2 of shape (N_f, B)
$\Delta W_1 = \sqrt{T/N_f} Z_1$, $\Delta W_2 = \sqrt{T/N_f}(\rho Z_1 + \sqrt{1-\rho^2} Z_2)$
Obtain exact GBM terminal values from the sum of ΔW_1
Obtain non-affine reference by log-state EM on the finest grid
for each coarse grid N **do**
    Sum groups of N_f/N fine increments; set $h = T/N$
    March price-space GBM EM and Milstein, or joint (X, Y) EM
    Record $|S_N - S_{\text{ref}}|$ and $\varphi(S_N) - \varphi(S_{\text{ref}})$
end for
Repeat independently over batches; retain each batch's whole grid vector
Estimate means, standard errors and bootstrap slopes; report reference sensitivity

Submission verification (7 October 2026). The portable `code/run_all.py` was executed end to end with the original fixed seeds; all 12 integrated checks passed. Eight additional analytic special-case tests passed in `code/validation_tests.py`. Raw evidence is included in `verification_run.log` and `validation_tests.log`. Tables below preserve the original recorded run; fresh timings are stored separately because wall time is machine dependent.

6 Results and interpretation

6.1 GBM: strong and weak convergence

Table 3: Table 3: GBM errors and signed mean biases. Entries with ± give pointwise 95% half-widths, not standard deviations. The last column applies to both price schemes.

N	EM strong	Milstein strong	EM mean bias	Milstein mean bias	Exact bias
4	3.0492 ± 0.0261	1.4880 ± 0.0121	-0.6205 ± 0.0347	-0.6002 ± 0.0159	-0.58965
8	2.1722 ± 0.0167	0.7677 ± 0.0062	-0.3210 ± 0.0234	-0.3062 ± 0.0082	-0.29999
16	1.5373 ± 0.0109	0.3893 ± 0.0032	-0.1565 ± 0.0160	-0.1550 ± 0.0042	-0.15132
32	1.0917 ± 0.0074	0.1956 ± 0.0016	-0.0805 ± 0.0111	-0.0774 ± 0.0021	-0.07600
64	0.7743 ± 0.0052	0.0982 ± 0.0008	-0.0423 ± 0.0079	-0.0388 ± 0.0011	-0.03808
128	0.5495 ± 0.0036	0.0492 ± 0.0004	-0.0203 ± 0.0055	-0.0194 ± 0.0005	-0.01906
256	0.3887 ± 0.0025	0.0246 ± 0.0002	-0.0120 ± 0.0039	-0.0097 ± 0.0003	-0.00954
512	0.2739 ± 0.0018	0.0123 ± 0.0001	-0.0050 ± 0.0028	-0.0049 ± 0.0001	-0.00477
1024	0.1946 ± 0.0013	0.0062 ± 0.0001	-0.0018 ± 0.0020	-0.0024 ± 0.0001	-0.00239

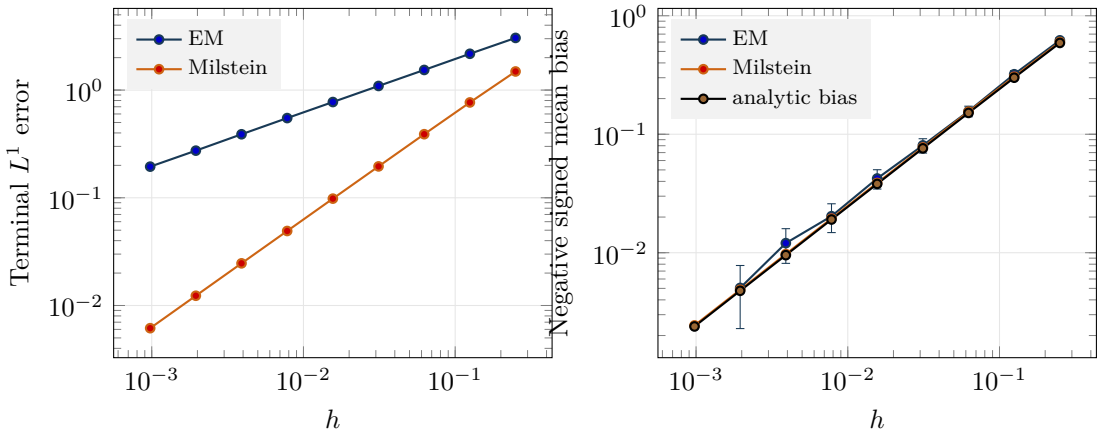


Figure 1: The strong convergence slopes differ noticeably, although both methods have the same first-moment bias in theory. The error bars show the 95% Monte Carlo intervals. For EM with $N = 1024$, the mean-bias interval includes zero, so it is not shown in this log-scale plot. Its signed value is reported in Table 3. The notebook uses a symmetric-log scale to display this interval.

The fitted strong convergence slopes, with 95% joint batch-bootstrap intervals, are 0.4975 [0.4952, 0.4994] for EM and 0.9972 [0.9966, 0.9978] for Milstein. These results are close to the expected rates of 0.5 and 1, respectively. The EM estimate does not have to include exactly 0.5 because the fitting uses a finite range of step sizes. In addition, the bootstrap reflects sampling variability but does not account for truncation errors in the asymptotic expansions. With errors of 0.194606 and 0.006157 at $N = 1024$, it is evident that retaining the quadratic variation is decisive for pathwise accuracy.

Regarding weak convergence, the exact bias curve yields a fitted slope of 0.9949. Empirically, the weak-mean slopes are 0.9610 [0.8999, 1.0466] for EM ($N \leq 256$) and 0.9948 [0.9938, 0.9957] for Milstein (all levels). The finest EM signed estimate is -0.001819 with a 95% half-width of 0.001953, which includes zero; therefore, there is no basis to claim a resolved nonzero bias or to blindly fit all EM points. While Milstein's analytic first-moment bias matches EM's, its paired mean estimator exhibits substantially lower variance.

6.2 Distribution, quantiles and analytic moments

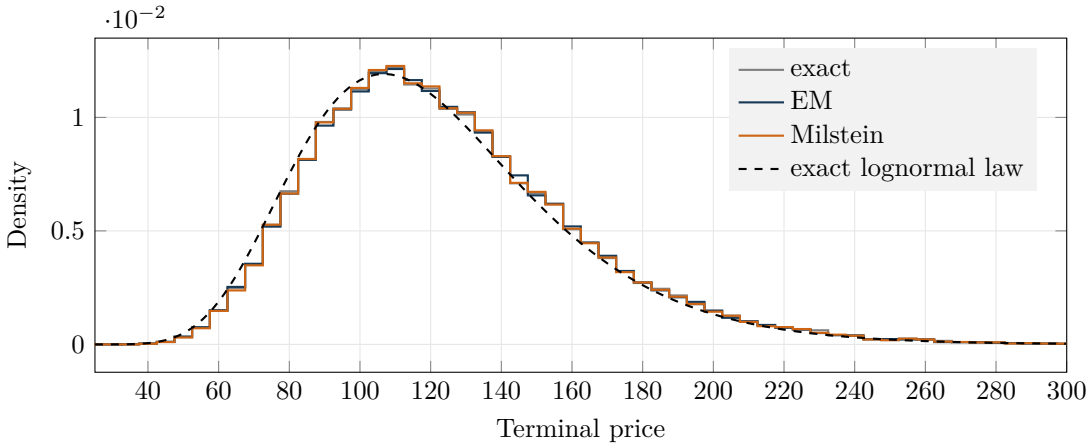


Figure 2: The histograms compare the price estimates from the different schemes with exact simulation at $N = 32$, using a bin width of 5. The analytic lognormal density is also included for comparison. For a clearer view, the plot only shows prices between 25 and 300. However, all 65,536 observations are included in the estimation, with no renormalization. It is worth noting that comparing marginal distributions alone is not enough to assess pathwise errors between coupled simulations.

Table 4: Quantiles for the center and tails. Even with exact simulation, finite-sample deviations persist.

p	Exact quantile	Exact simulation	EM ($N = 32$)	Milstein ($N = 32$)
0.01	58.106	58.032	57.644	58.280
0.05	71.287	71.348	71.312	71.565
0.50	116.766	116.740	116.877	116.724
0.95	191.259	192.508	191.771	191.907
0.99	234.645	235.721	233.999	234.685

The theoretical mean is 122.1403, compared with 122.3222 from exact simulation, with a standard error of 0.1473. The sample variance is 1421.9593, which is relatively close to the theoretical value of 1404.9152. We also calculated the average of $(S_T - \mathbb{E}S_T)^2$, obtaining 1421.9707 with a standard error of 10.4649. Since the calculation uses the theoretical mean, this estimator is unbiased for the exact variance. The differences between the simulated and theoretical results are consistent with Monte Carlo sampling variation. Although histograms help us compare the overall distributions, they may not show errors along individual coupled paths. The reported quantiles are also sample-based estimates, so they should not be treated as exact values or formal goodness-of-fit measures.

6.3 Non-affine reference validation and strong convergence

The increment sample contains 134,217,728 pairs. For $h_f = 1/4096$, the two empirical variances are 0.0002441197 and 0.0002441035, both close to the theoretical value of 0.000244140625. The correlation is -0.6999956, which is also close to the theoretical value of -0.7. All simulated price-factor states stay within the predefined limits, keeping the exponentiated prices strictly positive. The terminal prices range from 25.1046 to 301.2824. These checks suggest that the numerical implementation is working as intended, although they do not establish that the model accurately represents real financial markets.

Table 5: Non-affine log EM relative to the $N_r = 4096$ EM reference. Signed payoff differences mostly fail to resolve a bias; one isolated significant point is not a weak-rate study. All ± values are 95% half-widths.

N	Strong discrepancy	Call-payoff difference	Bias/SE
8	1.25305 ± 0.01224	0.00864 ± 0.01574	1.08
16	0.89935 ± 0.00846	0.00884 ± 0.01112	1.56
32	0.63671 ± 0.00593	0.01093 ± 0.00786	2.73
64	0.44464 ± 0.00413	0.00439 ± 0.00547	1.57
128	0.31468 ± 0.00289	0.00098 ± 0.00387	0.50
256	0.22095 ± 0.00202	-0.00004 ± 0.00272	-0.03

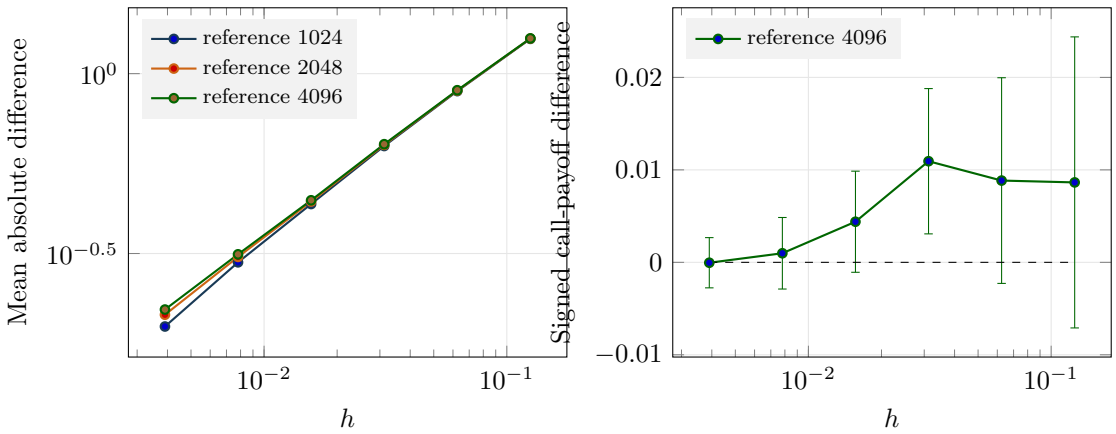


Figure 3: Nested-grid log EM supports an empirical half-order strong regime, with visible reference sensitivity at small coarse steps. The weak panel is signed and uses a linear vertical scale: intervals crossing zero are displayed rather than converted into spurious log errors. Bars show sampling uncertainty only.

Using reference sizes $N_r = 1024$, 2048, and 4096, the estimated convergence rates are 0.5170 [0.5130, 0.5212], 0.5058 [0.5021, 0.5097], and 0.5003 [0.4966, 0.5046], respectively. When the reference size increases from 2048 to 4096, the rate changes by 0.0054, while the strong discrepancy at $N = 256$ changes by 3.28%. Both results meet the operational stability checks. However, the finest coarse-grid point is still affected by the choice of reference, since the shift between references is larger than the Monte Carlo interval. For this reason, we present the empirical rates and discrepancies relative to a finite reference rather than treating them as exact strong errors.

Table 6: Reference refinement is a separate source of uncertainty. A small signed weak shift can coexist with a much larger absolute pathwise difference.

Reference pair	Mean absolute price difference	Signed payoff difference
1024-2048	0.080764 ± 0.000751	-0.000359 ± 0.001005
2048-4096	0.056758 ± 0.000522	0.000308 ± 0.000699
1024-4096	0.098672 ± 0.000919	-0.000051 ± 0.001226

The adjacent-reference strong differences decrease by a ratio 1.423, near $\sqrt{2}$, consistent with a half-order regime. Their nonzero size matters: the 2048–4096 difference is about 25.7% of the $N = 256$ discrepancy. The triangle inequality implies

$$|\mathbb{E}|S_h - S| - \mathbb{E}|S_h - S_r|| \leq \mathbb{E}|S_r - S|, \tag{44}$$

but the right side is unknown. Neither an adjacent-grid difference nor a Richardson extrapolation is a rigorous upper bound without additional assumptions. The reference hierarchy is sufficient for the stated empirical half-order conclusion, not for certifying four-digit exact pathwise error.

6.4 Weak-payoff uncertainty and a more precise expected payoff

For the finest reference, the estimated expected payoff is 14.57837 (SE 0.11815), and the estimated value of $\mathbb{E}[S_{N_r}]$ is 105.10376 (SE 0.16795), which is close to the analytical value of 105.12711. However, although this agreement is expected from (28), it does not tell us how large the weak error in the call payoff is.

In Table 5, the confidence intervals for most paired call-payoff differences include zero. Since the results do not show a clear and statistically significant power-law pattern, we do not estimate an empirical weak convergence order for this quantity. Also, because the payoff function has a kink,

theorems for smooth test functions cannot be used directly without checking the required regularity conditions. More simulation paths, variance reduction for payoff differences, or a multilevel estimator may help identify a small weak bias.

Using an independent pilot sample, the control coefficient is estimated as $\hat{\beta} = 0.6387$, reducing the evaluation-sample variance by a factor of 6.23. On the same subset, the raw payoff estimate is 14.64582 (SE 0.12637), while the controlled estimate is 14.58551 (SE 0.05062, 95% CI [14.48629, 14.68472]). The control therefore gives a more precise estimate of the fine-grid expected payoff. However, its confidence interval has a half-width of about 0.10, which is much larger than the observed changes between adjacent reference levels. This comparison helps assess the available numerical resolution, but it does not provide a theoretical bound on the continuum price bias.

6.5 Positivity counterexample

Table 7: Analytic one-step negativity probabilities and observed counts in 200,000 independent updates per row. Stress: $\mu = 0.05$, $\sigma = 1.5$; baseline: $\mu = 0.2$, $\sigma = 0.3$.

Case	h	p_{EM}	EM count	p_{Mil}	Mil. count
baseline	1.0	3.17e-05	4	0.00000	0
baseline	0.5	1.08e-07	0	0.00000	0
baseline	0.25	1.28e-12	0	0.00000	0
baseline	0.125	2.15e-22	0	0.00000	0
stress	1.0	0.242	48525	0.43569	87523
stress	0.5	0.167	33324	0.13192	26317
stress	0.25	0.0885	17527	0.00000	0
stress	0.125	0.0289	5839	0.00000	0

Zero observed failures do not prove zero probability: with 200,000 trials, the approximate 95% upper bound after zero events is $3/M \simeq 1.5 \times 10^{-5}$. The baseline Milstein zero probability follows from the quadratic minimum, not the experiment. At the stressed $h = 1$, Milstein actually has a *larger* one-step negativity probability than EM. This falsifies the claim that adding a Milstein correction always improves positivity. All positivity stress parameters are labelled separately from the main convergence benchmark.

6.6 Extension: preserved moments, limited price advantage

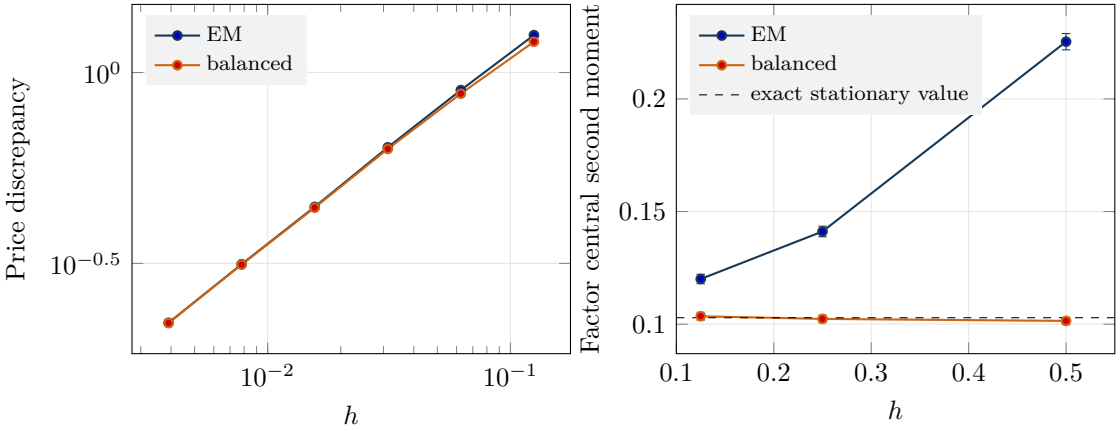


Figure 4: The extension has little effect on fine-grid terminal price error (left), but preserves the exact factor stationary variance across stable coarse steps (right). The latter uses the separate $T = 40$, $Y_0 = \theta$ stress IVP. The unstable EM $h = 1$ row is reported numerically in Table 8 rather than compressing this axis.

The balanced scheme exhibits an empirical strong price convergence slope of 0.4872 [0.4835, 0.4908] for $N = 8$ to 128 when compared against the EM reference. At $N = 8$, the error is 1.20398 for the balanced scheme versus 1.25305 for EM; at $N = 256$, these values are 0.22063 and 0.22095, respectively. This minor improvement on the coarse grid does not alter the underlying half-order convergence. Furthermore, at $N = 8$, the balanced scheme's payoff difference from the reference is 0.02835 ± 0.01509, compared to 0.00864 ± 0.01574 for EM, demonstrating that preserving a factor moment does not necessarily yield a smaller price-payoff bias.

For the factor-only stress experiment, we initialize the factor at $Y_0 = \theta = -0.2$ and set $T = 40$ with fixed model coefficients. This isolates the exact centered recurrence (36), distinct from the original one-year IVP, and utilizes 32,768 paths with step sizes $h = 1, 1/2, 1/4, 1/8$.

Table 8: Factor stress test, $Y_0 = \theta$, $T = 40$. All uncertainties are nominal 95% half-widths; the unstable EM $h = 1$ interval is unreliable and fails against the exact discrete moment. Stable rows test variance accuracy as well as boundedness.

Method	h	A_h	Sample central second moment	Analytic $T = 40$ moment
EM	1.0	1.3600	11735.9645 ± 8448.8360	228342.9966
balanced	1.0	0.1067	0.1027 ± 0.0016	0.1029
EM	0.5	0.1800	0.2254 ± 0.0036	0.2283
balanced	0.5	0.2132	0.1014 ± 0.0016	0.1029
EM	0.25	0.3400	0.1411 ± 0.0024	0.1418
balanced	0.25	0.4248	0.1023 ± 0.0017	0.1029
EM	0.125	0.6075	0.1201 ± 0.0021	0.1192
balanced	0.125	0.6419	0.1035 ± 0.0018	0.1029

At $h = 1$, standard EM has $A_h = 1.36 > 1$. Starting at $U_0 = 0$, its exact discrete second moment after n steps is

$$v_n = \xi^2 h(1 + \theta^2)\frac{A_h^n - 1}{A_h - 1}. \tag{45}$$

For $n = 40$, the theoretical value is 228343.00, but the simulated sample mean is only 11735.96, which is about 5.1% of the exact value. The usual normal confidence interval does not contain the theoretical value either. This happens because the expected value is strongly affected by rare

paths with very large values. With a finite sample, these extreme paths may not appear often enough, so both the sample mean and the estimated standard error can be misleading. This is not a contradiction of the stability analysis. Instead, it shows a limitation of Monte Carlo estimation in this setting. The deterministic recurrence still confirms that EM is unstable, while the simulation shows that a small estimated error does not always mean the result is accurate. For this reason, we cannot claim reliable 95% coverage for this stress test.

For the balanced scheme, the variances stay close to 0.102857 across all tested values of h, which agrees with Proposition 1. At the coarse step size $h = 1/2$, EM gives a stationary variance of 0.228293, more than twice the exact value, although the scheme is still stable. This shows that a stable numerical method does not necessarily reproduce the correct variance or give accurate price estimates. These properties need to be considered separately.

6.7 Measured cost at a common accuracy target

We measure integration cost using the median running time from seven runs after a warm-up, with 8,192 paths and pre-generated Brownian increments. The number of path updates is also reported. The recorded times include the calculations within each solver, memory allocation and any additional stability checks. However, they do not include random number generation, input preparation, library imports, reference solutions, bootstrap calculations, plotting or animation. For the performance comparison, we require the terminal mean-absolute price error to be no greater than 0.5. We use the upper bound of the sampling confidence interval to check this requirement, rather than relying only on the estimated mean error. For each method, we then choose the largest step size that meets this target.

Table 9: Measured integration cost for 8,192 paths at a common 0.5 error ceiling. GBM uses the exact oracle; non-affine errors are relative to $N_r = 4096$. The displayed intervals are sampling intervals.

Model	Method	N	Error estimate	Time (ms)	Path updates
GBM	EM	256	0.3887 ± 0.0025	1.4243	2,097,152
GBM	Milstein	16	0.3893 ± 0.0032	0.1748	131,072
GBM	exact	1	0 (exact)	0.0432	8,192
Non-affine	EM	64	0.4446 ± 0.0041	6.5389	524,288
Non-affine	balanced	64	0.4425 ± 0.0041	6.4562	524,288

For GBM, EM and Milstein give almost the same error at their selected step sizes, so their computational costs can be compared under similar accuracy requirements. In this experiment, Milstein uses 16 times fewer path updates and runs about 8.15 times faster than EM. Exact terminal simulation is still the best choice for GBM, since it has no discretization error and only needs one Gaussian increment. However, this comparison only applies to terminal values. If the full price path is needed, the methods would have to be compared on the same output grid, which would involve additional computation.

For the non-affine models, both methods use $N = 64$ to meet the same reference-error requirement. Their running times are quite similar, and the small differences do not provide enough evidence that one method is generally faster. The measured errors are both around 0.44, but these are errors relative to the numerical reference, not the exact solution. The balanced method therefore does not show a clear computational advantage for the one-year price simulation. Its main benefit is that it preserves the factor moments more accurately, rather than reducing the overall simulation cost.

6.8 Transient visualisation

The notebook includes a 65-frame animation showing six non-affine sample paths over one year. Price and instantaneous volatility are displayed in two aligned panels. The paths are generated using a fixed random stream, making the results reproducible. The animation shows how volatility changes with the underlying factor, while prices remain continuous and positive throughout the simulation. However, the animation is mainly used to illustrate the behaviour of the model, rather than to test convergence. Six sample paths are not enough to represent the full distribution. For the statistical results and terminal error analysis, we therefore use a much larger number of independent simulated paths.

6.9 Guide-compliant rate overlays and cost comparison

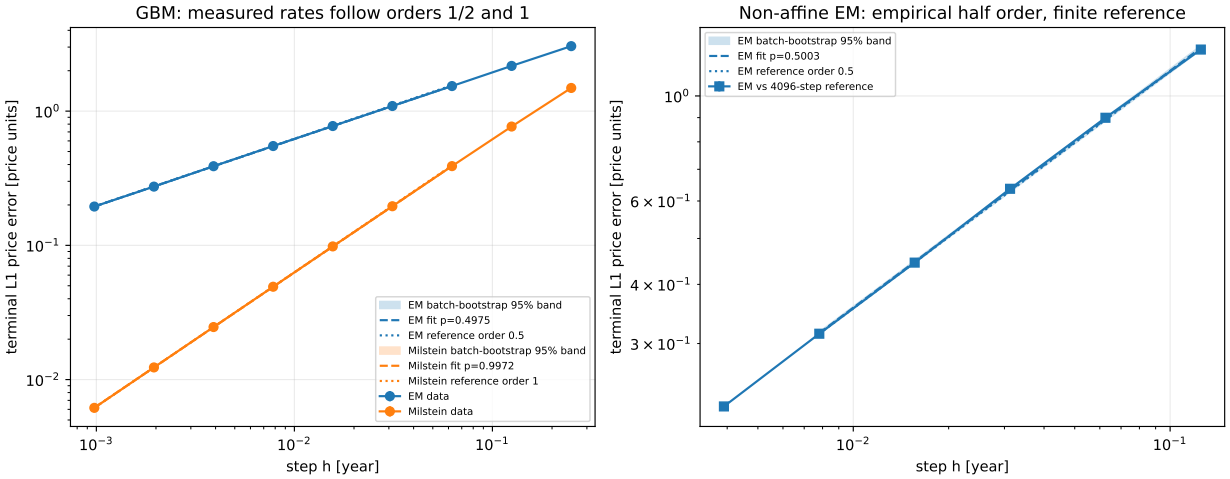


Figure 5: Strong error against step size in years, with theoretical reference slopes and fitted lines. Shaded regions are pointwise 95% bootstrap sampling bands from 500 resamples of independent batches; each resample retains its entire vector across grids. GBM uses $N = 16, \ldots, 1024$ for fitting. The non-affine fit uses $N = 8, \ldots, 128$ against the 4096-step reference; the displayed $N = 256$ point is excluded from the fit. Bands exclude reference and asymptotic-fit bias, and can be visually narrow at this sample size. All values are regenerated by the companion script.

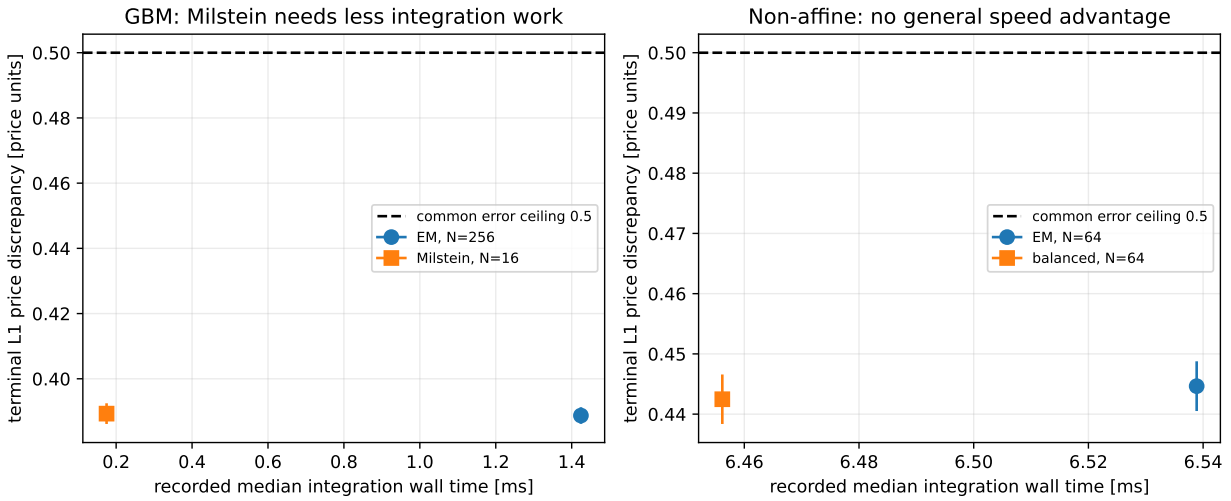


Figure 6: Recorded median integration wall time versus achieved terminal L^1 discrepancy at the common ceiling 0.5. Vertical intervals are pointwise sampling intervals. Each point is the least costly tested admissible grid, rather than an equal-step comparison. GBM uses the exact pathwise oracle; the non-affine model uses the finite reference. Times are the archived measurements quoted in Table 9, on 8192 paths after warm-up, with seven repeats; RNG, setup, reference construction, plotting and bootstrap cost are excluded. Exact GBM has zero time-discretisation error and is discussed separately.

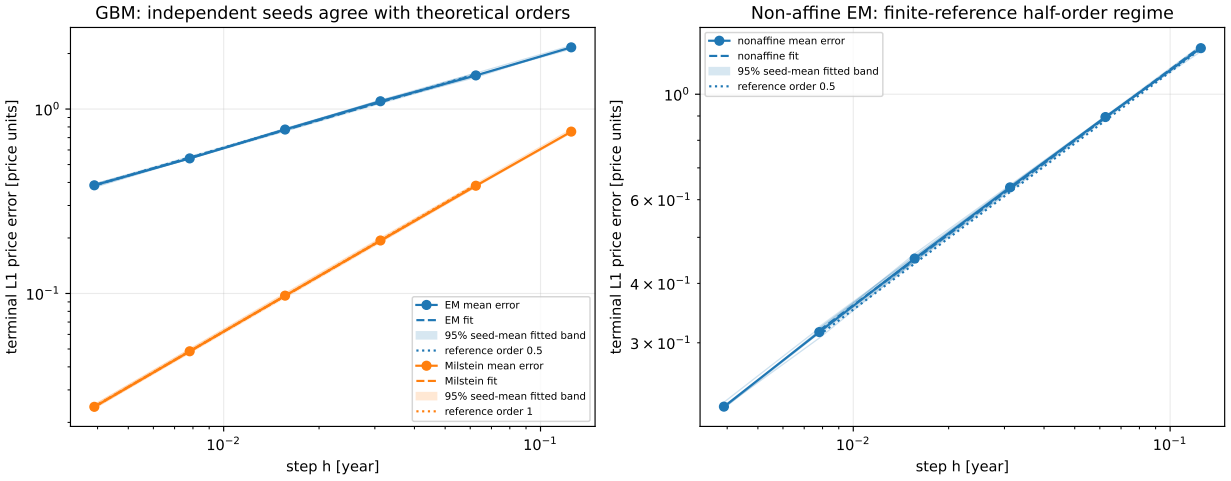


Figure 7: Supplementary whole-experiment repetition over five independent seeds (20261001–20261005), each with 4096 paths and a 4096-step fine grid. Thin lines show seed-specific errors; the fitted mean and an approximate Student-t 95% band use the spread of the five fitted log-error curves (four degrees of freedom). GBM fits use $N = 16, \ldots, 256$; non-affine fits use $N = 8, \ldots, 128$. This checks seed sensitivity separately from the larger main run's batch bootstrap. It is a confidence band for the mean fitted log-error, not a prediction band for a new seed, not simultaneous coverage, and not a bound on reference bias.

Five seeds are a limited supplementary replication, not a substitute for a sufficiently large path budget. The exact per-seed errors and slopes are recorded in `outputs/independent_seeds.json`; the main inference still relies on the larger experiment, analytic checks and reference refinement.

7 Limitations, conclusions and practical recommendations

The GBM experiments show the expected strong convergence orders, together with a first-order mean bias and consistent error distributions. For this model, the exact transition is therefore the most practical option. In the non-affine model, where an exact pathwise solution is not available,

the nested refinement results suggest a convergence rate close to one-half. This result should still be treated with some caution, since the estimated errors depend on the numerical reference used. In particular, narrow confidence intervals do not necessarily mean that the true discretization error is known accurately. The weak-payoff differences are also difficult to distinguish from Monte Carlo noise, so the results are not sufficient to estimate a reliable weak convergence order.

For the factor process, we introduced a moment-balanced update and showed that it preserves the exact mean and stationary variance. This removes the second-moment step-size restriction associated with EM for this particular factor. However, the numerical results do not show a clear improvement in the computational cost of one-year price simulations, nor does the method consistently produce smaller payoff biases. The large-step experiment also shows a practical difficulty with Monte Carlo estimation. When a small number of rare paths contribute heavily to a moment, even a relatively large sample may fail to capture their effect. In such cases, the usual confidence intervals can be misleading.

There are several limitations to these results. All parameters are synthetic, and the methods have not been tested using market calibration or hedging experiments. The strong-error analysis considers terminal L^1 errors rather than errors over the entire simulated path. In addition, the confidence intervals for the fitted convergence slopes only account for the statistical variation in the regression and do not include possible errors from the assumed asymptotic behaviour or the numerical reference. For the non-affine model, the reference solution is constructed using a refined EM scheme rather than an independent exact method. The exact moment calculations and frozen-factor comparisons provide useful checks, but they cannot fully verify the coupled model. The control variate reduces Monte Carlo variance, although it does not remove the discretization bias in the continuous-time price. The unstable stress test also illustrates why normal confidence intervals should not be trusted when extreme outcomes are poorly represented in the sample. We do not claim methodological novelty, particularly since more advanced methods such as multidimensional Milstein schemes and adaptive Brownian-bridge integrators were not implemented.

In practice, exact GBM simulation is useful for checking an implementation before moving to more complicated models. Simulating the log-price is also preferable when price positivity needs to be maintained. For convergence tests, using the same Brownian paths across different step sizes makes comparisons more informative. Weak-payoff differences should be reported together with their Monte Carlo uncertainty, and the numerical reference should be refined before drawing conclusions about convergence rates. Finally, when accurate financial payoffs are required, sampling error and time-discretization error need to be considered separately. Reporting more decimal places does not, by itself, make a numerical estimate more accurate.

References

[1] Course team. *Project 1 Brief: Numerical Solution of ODE Initial Value Problems*. Supplied course material, Weeks 1–4, 2026. File: `ODE_IVP_Project_Brief.pdf`.

[2] Course team. *Problem Pack: Five Suggested Directions for the ODE IVP Project*. Direction (4): Financial SDE, GBM benchmark and non-affine volatility. Supplied course material, 2026. File: `problem_pack.pdf`.

[3] Course team. *Assessment Rubrics for PBL Numerical PDEs*. 11 September 2026. Supplied course material. File: `Numerical_PDEs_Assessment_Rubrics.pdf`.

[4] D. J. Higham. An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM Review*, 43(3):525–546, 2001. doi:10.1137/S0036144500378302. Bibliographic existence verified against the publisher and author-affiliated repository; foundational source, not a source of the present experiment's numerical results.

[5] P. E. Kloeden and E. Platen. *Numerical Solution of Stochastic Differential Equations*. Springer, 1992. doi:10.1007/978-3-662-12616-5.

[6] C. R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi:10.1038/s41586-020-2649-2.

[7] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.

A Part A – Team AI Transparency Log

Team D. One collective log for the team. This report, its mathematical exposition, code and numerical analysis were prepared with AI assistance. Experiments are synthetic simulations. The entries distinguish analytic checks and executed computational evidence from personal review; members must confirm their understanding before signing A4. Routine assistance included array coding, document formatting and rendered-output checks.

A1. AI tools used

Tool	Version / Date	Primary use
OpenAI Codex	September–October 2026; precise model version not archived	Mathematical exposition, code and experiment preparation, claim checks, report/slide revisions and submission organisation.

A2. Significant AI-assisted claim audits

Three material claims are audited below; routine edits are summarised above. Original project conversation records are retained separately. Members should correct the entries if their actual work adds or changes evidence.

Item	Prompt / claim / assumptions	Risk and possible falsifier
1	**Claim:** scalar Milstein has strong order one on price-space GBM. Assumptions: common Brownian paths, smooth scalar coefficients and finite moments. The task requested the Itô–Taylor derivation and comparison with EM.	Log-GBM removes the correction, while independent coarse/fine noise invalidates strong comparisons. An incorrect correction or coupled slope far from one would challenge the implementation.
2	**Claim under audit:** the non-affine model has a validated empirical weak order one. Assumptions needed: a resolved reference, observable bias above sampling uncertainty and careful treatment of the nonsmooth payoff.	Absolute noisy sample means can create a spurious slope. Intervals crossing zero or reference-sensitive bias would prevent a rate conclusion.
3	**Claim under audit:** the moment-balanced update is stable and therefore always better. Assumptions: $2\kappa > \xi^2$, fresh Gaussian increments and the specified explicit factor update. Stability here concerns factor second moments.	Stationary-variance stability does not establish exact prices, higher price order or lower cost. Coupled price/cost results can falsify a general superiority claim.

A3. Independent verification and decision

These checks use analytic special cases and recorded computations, not AI output as its own oracle. Computational checks were executed during the assisted preparation; this does not assert that every member personally ran or independently reviewed them.

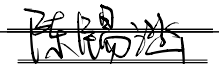
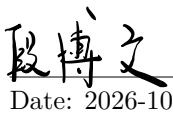
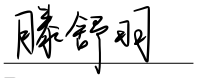
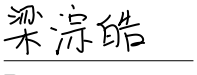
Item	Independent check and evidence location	Final decision / modification / limitation
1	Derive the scalar correction and compare on common paths with the exact exponential. Check transition composition, zero diffusion and the one-step formula. Evidence: Eqs. (4)–(21), Fig. 1, `code/validation_tests.py`, `validation_tests.log`.	Supported for this GBM benchmark: slope 0.9972. Keep the price-space experiment. Do not transfer scalar order one to the joint model; Eq. (29) shows noncommuting diffusion fields.

Item	Independent check and evidence location	Final decision / modification / limitation
2	Compare signed coupled payoff differences and their intervals on three reference grids; check the known mean separately. Evidence: Tables 5 and 6, `outputs/results.json`, `verification_run.log`.	Reject a validated empirical weak-rate claim at this sample size. Report finite-reference strong convergence and unresolved weak biases. Control variates reduce sampling variance, not reference bias.
3	Derive the factor moment recurrence and stationary variance; test the fixed-point identity. Compare stress moments, coupled prices and matched-error cost. Evidence: Proposition 1, Table 8, Fig. 4, Table 9, `code/validation_tests.py`.	Accept the precise factor mean/variance stability claim; reject universal price or speed superiority. The stressed EM interval is unreliable when rare-event tails are undersampled.

A4. Intellectual ownership declaration

Each member identifies their actual contribution and signs their own row. Signing affirms that they can explain and modify the team-owned numerical methods, justify the claims, and state assumptions and tests at trusted library interfaces without AI during the viva. Personal signatures and understanding are not inferred here.

Team-owned artefacts: directly implemented updates, Brownian coupling, error estimators, factor-moment proofs, tests and interpretation. **Trusted dependencies:** NumPy array arithmetic and random generation; SciPy sigmoid/normal functions; Matplotlib plotting; Pillow GIF export. Codex assistance is disclosed in A1 and is not a numerical-library interface.

Name	Signature / date	Team-owned artefact / contribution	Trusted dependency and interface
Xihan Chen 2360253	 Date: 2026-10-11	Report Secs. 1 and 5; EM/Milstein convergence slides: error definitions, uncertainty and reproducibility.	NumPy random-number generation: independent paths and shared Brownian increments across grid levels; documented variance and correlation checks.
Bowen Duan 2362211	 Date: 2026-10-11	Non-affine SDE explanation; joint EM/Milstein distinction; nested Brownian grids and control-variate interpretation (Secs. 2.3, 3.4–3.5, 5.1, 5.4).	NumPy Gaussian increments require variance h and correlation ρ; SciPy `expit` gives bounded logistic volatility. Relevant checks are in A3; personal verification must be confirmed before signing.
Shuyu Teng 2362204	 Date: 2026-10-11	Numerical scheme derivation, stability and moment-balanced scheme analysis (Secs. 3, 4).	Dependencies: NumPy, SciPy. Assumptions on Brownian increments; stability and positivity numerical checks.
Conghao Liang 2362212	 Date: 2026-10-11	Project manager: scope, presentation review, revision decisions and submission organisation.	Team dependency boundaries listed above; personally confirm understanding before signing.

B Part B – Individual Contribution Statements (ICS)

Four individual records for Team D, one page per member. All four statements are included with member names, student IDs and signatures. Each member should confirm the final wording of their own statement.

Individual Contribution Statement – Project 1 (ODE IVP) – Xihan Chen

Team: D **Name:** Xihan Chen **Student ID:** 2360253

Role this project: Report Writing and Convergence Analysis Presentation

The specific things I did this project:

1. I wrote the research background and objectives, explaining why geometric Brownian motion provides an exact benchmark and why the non-affine volatility model requires numerical references. I clarified the scope of the investigation and its focus on reliable numerical simulation.

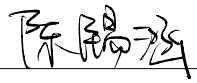
2. I prepared the presentation content on strong and weak convergence for Euler-Maruyama and Milstein. I explained shared Brownian paths, terminal mean absolute errors, signed mean biases and convergence-rate estimation. I organised the material from definitions to calculations and results, including the influence of Monte Carlo uncertainty.

3. I documented the experimental design and reproducibility procedures, including nested Brownian grids, confidence intervals, bootstrap slope estimates and reference-refinement checks. I explained the separation of discretisation error from sampling uncertainty, the purpose of control variates, and the reported random seeds, implementation choices and validation checks.

What I would do differently:
I would simplify the convergence explanations earlier and use one numerical example to connect the formulas with the graphs.

Anything I should have asked for help with earlier:
I should have clarified strong versus weak error and the interpretation of confidence intervals earlier.

Evidence / Part A cross-reference:
The submitted presentation and report, and the recorded discussion of convergence definitions and calculations. Related AI audits: Part A, A2-A3, items 1-2. This statement describes writing and presentation work, not personal execution of the reported code tests.

Signature:  **Date:** 2026-10-11

Individual Contribution Statement – Project 1 (ODE IVP) – Bowen Duan

Team: D **Name:** Bowen Duan **Student ID:** 2362211

Role this project: Mathematical Theory

The specific things I did this project:

1. I explained the non-affine stochastic-volatility model prescribed in the Problem Pack, including the log-price equation, mean-reverting factor, bounded logistic volatility and correlated Brownian drivers. This explanation corresponds to report Section 2.3.

2. I explained the use of Euler–Maruyama for the joint system and why the scalar Milstein formula cannot be transferred directly. The noncommuting diffusion fields require cross terms and iterated stochastic integrals for a full multidimensional Milstein method. I also explained nested Brownian grids for coupled comparisons (Sections 3.4–3.5 and 5.1).

3. I explained the control-variate construction for the expected call payoff, including the known stock-price mean, the separate pilot sample and the evaluation sample. I clarified that variance reduction improves sampling precision but does not remove time-discretisation bias (Sections 5.4 and 6.4).

What I would do differently:
I would reduce unnecessary theorems and organise the explanation around the model, numerical method and interpretation of results.

Anything I should have asked for help with earlier:
I should have asked earlier for a clearer explanation of multidimensional noise and the distinction between sampling variance and discretisation bias.

Evidence / Part A cross-reference:
Evidence: the report sections listed above and the supplied personal statement. Related AI audits: Part A, A2–A3, items 1–2. My contribution here concerns theoretical explanation; I do not claim personal execution of the code tests.

Signature:  **Date:** 2026-10-11

Individual Contribution Statement – Project 1 (ODE IVP) – Shuyu Teng

Team: D **Name:** Shuyu Teng **Student ID:** 2362204

Role this project: Report Writing and Numerical Defects Presentation

The specific things I did this project:

1. I did the writing work of section 3, which is about numerical methods and convergence concepts. And I completed the derivation for scalar Milstein correction term, exact weak-error formula for GBM and correlated Brownian increment construction. I also explained why scalar Milstein can't be directly applied to the two dimensional non-affine stochastic-volatility model through diffusion commutator argument in the paragraph.

2. I did the writing work of section 4, which is about the stability, positivity and a derived extension. I derived one-step negativity-probability formulas for both EM and Milstein method. I inferred that the second-moment recurrence of the stochastic-volatility factor using standard EM and built up the moment-balanced factor scheme together for the exact conditional mean and stationary variance in period.

3. I performed mathematical sanity-checks linking theory to numerical experiments and I checked that the log-price EM martingale property and find out limitations within the report text. In the presentation I was responsible for discussing the comparison between EM, Milstein sample distributions and the true logarithmic normal distribution and the positivity-negative price issue, which related to what I responsible for in the report.

What I would do differently:
I would separate theoretical derivations and numerical sanity-check work earlier.

Anything I should have asked for help with earlier:
I should agree with teammates on the symbols variables at very beginning cause this saves time and effort comparing to change these to the same in the end.

Evidence / Part A cross-reference:
Report source `financial_sde_report.tex` (Section 3, Section 4) and corresponding presentation materials. Related AI audits: Part A, A2-A3, items 1-3.

Signature:  **Date:** 2026-10-11

Individual Contribution Statement – Project 1 (ODE IVP) – Conghao Liang

Team: D **Name:** Conghao Liang **Student ID:** 2362212

Role this project: Project Manager

The specific things I did this project:

1. I set the project scope around the GBM benchmark and non-affine volatility model. I supplied the course materials and requested checks against the rubric, report template and submission requirements. I organised the report, code package and presentation as separate deliverables and arranged suggested responsibilities for the four members.

2. I reviewed the presentation and directed revisions to `financial_sde_presentation.tex` and `financial_sde_speaker_script.md`. I requested explicit Itô and EM/Milstein formulas, clearer strong and weak convergence calculations, distribution comparisons and negative-price conditions. I also identified a gap in the transition to stochastic volatility and requested explanations connected to the model equations. I adjusted slide layout, content and length through successive reviews.

3. I used OpenAI Codex to assist with mathematical exposition, code generation, experiments and document preparation. I requested limitations, verification evidence and an AI Transparency Log. My contribution was scope management, critical questioning and revision decisions; I do not claim independent implementation of `gbm` or `sv`, or personal execution of the recorded tests.

What I would do differently: I would establish the report structure and presentation timing earlier, and record review decisions alongside revisions.

Anything I should have asked for help with earlier: I should have clarified convergence definitions and the distinction between analytic results and simulation estimates earlier.

AI audit cross-reference: Report Appendix A, especially Audit 1 (scalar Milstein) and Audit 2 (weak-rate interpretation). The assistance used was OpenAI Codex in the recorded project conversation.

Evidence: Recorded requests and revisions in the project conversation; `financial_sde_presentation.tex`, `financial_sde_speaker_script.md`, and the final submission package. No Git commit history is claimed.

Signature: 梁淙皓 **Date:** 2026-10-11

Prepared from the recorded project conversation. The member must review this statement, confirm or adjust the reflections and sign personally. No personal signature or Git history is inferred.

C Reproducibility and submission instructions

Extract `project1_code.zip`; from its `code_package` folder run:

```
python -m pip install -r requirements.txt
python code/validation_tests.py
python code/run_all.py
```

Use Python 3.10 or newer. The full run regenerates JSON tables, all static figures, rate bands, cost figures and the path animation. Paths are anchored to the script's folder, not to a particular computer. No market data or network access is required after dependency installation. The README states seeds, batch sizes, dependency bounds, timing exclusions and how archived and fresh outputs differ.

The fresh submission run used Python 3.12.14, NumPy 2.5.3, SciPy 1.18.1, Matplotlib 3.11.2 and Pillow 12.3.0; the executable log is retained. The 12 integrated checks and eight separate analytic tests passed. The deliberately unstable stress case remains a reported negative result. A standard confidence interval is not asserted to have reliable coverage when the relevant tail is undersampled.

In the editable source ZIP, compile the root-level `project1_report.tex` using Tectonic or pdfLaTeX twice. Keep its `code_package` folder alongside the root-level main source. The local working copy uses a `sources` directory with relative paths adjusted accordingly. It embeds the main Python script using `filecontents*`, so compile in a dedicated directory. Helper/test listings and the supplementary figures are relative external files included in the bundle. The bibliography is inline. The submission consists of report PDF, code ZIP and the 17-page slides PDF; the source bundle is supplied separately for editing, not as a replacement for those three files.

Declarations. All data are synthetic, with no participants or private market datasets. No funding or conflict information was supplied; members must confirm their own declarations. AI use is disclosed in Appendix A. Names, contributions, signatures and historical evidence must be completed from authentic team records.

D Presentation and viva preparation

The companion PDF preserves the approved 17 slides. Suggested ten-minute allocation: slides 1–3 (1.5 minutes, motivation and exact solution); slides 4–5 (1 minute, retained Itô–Taylor terms); slides 6–9 (2 minutes, strong/weak calculations and results); slides 10–11 (1 minute, distributions); slides 12–13 (1 minute, positivity); slides 14–15 (1.5 minutes, non-affine model and multidimensional limitation); slides 16–17 (2 minutes, coupled references and factor/control-variate extension). Rehearsal is required to check this estimate. Detailed proofs and additional experiments belong in this report. At the user's direction, separate closing and backup slides were removed; the presenter should close slide 17 verbally with the main result and one limitation.

Cross-role questions everyone should answer. (1) Why must fine Brownian increments be summed rather than regenerated? Because strong error compares approximations on the same driving trajectory. (2) Why can both methods have the same weak mean bias but different strong orders? The correction is centred but captures a pathwise fluctuation. (3) Why is a correct mean insufficient for the non-affine solver? Log EM preserves that moment at every step size. (4) Why is the finest reference not exact? It is still an EM discretisation, and adjacent-reference changes are nonzero. (5) Why not use scalar Milstein twice? The diffusion fields do not commute. (6) What did the new scheme actually improve? The factor mean and stationary variance/stability, not a proved price order or general speed advantage. (7) Why did the stress interval fail? Rare events dominated the expectation, so the sample variance itself was unreliable. (8) What cost was excluded? RNG, reference construction, setup and plotting; the timed object was integration on fixed input arrays.

E Complete executable computational source

The first listing is the main executable source supplied in `code/run_all.py`. The code ZIP also supplies its importable kernels, the supplementary visualisation helper and the analytic checks. All are listed below so that the numerical and validation procedures are reviewable.

```
# %% Setup and reproducibility
from pathlib import Path
import sys, platform, time, json, math
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.special import expit, ndtr, ndtri
OUT = Path(__file__).resolve().parents[1] / 'outputs'; OUT.mkdir(exist_ok=True)
plt.rcParams.update({'font.size':10, 'axes.grid':True, 'grid.alpha':.25,
                     'figure.dpi':160, 'savefig.dpi':250, 'savefig.bbox':'tight'})
SEED=20260916
M_GBM=65536; M_SV=32768; BATCH=1024
RESULTS={'seed':SEED, 'python':sys.version.split()[0], 'numpy':np.__version__,
         'platform':platform.platform(), 'M_GBM':M_GBM,'M_SV':M_SV}
def summary(x):
    x=np.asarray(x); m=float(x.mean()); se=float(x.std(ddof=1)/np.sqrt(x.size))
    return {'mean':m,'se':se,'lo':m-1.96*se,'hi':m+1.96*se}
def slope_ci(batch_errors, hs, seed=17):
    # Resample independent path batches; retain dependence across levels.
    a=np.asarray(batch_errors); hs=np.asarray(hs)
    fit=lambda v: np.polyfit(np.log(hs),np.log(v),1)[0]
    rng=np.random.default_rng(seed)
    slopes=[fit(a[rng.integers(len(a),size=len(a))].mean(axis=0)) for _ in range(500)]
    return [float(fit(a.mean(axis=0))), *np.quantile(slopes,[.025,.975]).tolist()]
def savefig(name):
    plt.tight_layout(); plt.savefig(OUT/(name+'.png')); plt.savefig(OUT/(name+'.pdf'))
    show_figure(plt.gcf()) if 'show_figure' in globals() else None
    plt.close()
def gbm(dW,h,method,mu=.2,sigma=.3,s0=100.):
    # Rows are time, columns independent paths; no Python loop over paths.
    if method=='exact': return s0*np.exp((mu-.5*sigma*sigma)*dW.shape[0]*h+sigma*dW.sum(axis=0))
    f=1+mu*h+sigma*dW
    if method=='Milstein': f=f+.5*sigma*sigma*(dW*dW-h)
    return s0*np.prod(f,axis=0)
def sv(d1,d2,h,mu=.05,kappa=2.,theta=-.2,xi=.6, balanced=False):
    x=np.full(d1.shape[1],np.log(100.)); y=np.zeros_like(x)
    decay=np.exp(-kappa*h)
    noise=xi*np.sqrt(-np.expm1(-2*kappa*h)/(2*kappa*h))
    for a,b in zip(d1,d2):
        g=.1+.4*expit(y)
        x += (mu-.5*g*g)*h+g*a
        if balanced:
            y=theta+decay*(y-theta)+noise*np.sqrt(1+y*y)*b
        else:
            y += kappa*(theta-y)*h+xi*np.sqrt(1+y*y)*b
        assert np.all(np.isfinite(y)) and np.all(np.abs(x)<700)
    s=np.exp(x)
    assert np.all(np.isfinite(s)) and np.all(s>0) and np.all(np.isfinite(y))
    return s,y
print('Reproducibility:', RESULTS)
# %% GBM: nested paths, strong and weak errors
ng=np.array([4,8,16,32,64,128,256,512,1024]); hg=1/ng
rng=np.random.default_rng(np.random.SeedSequence([SEED,1]))
gerr={k:[] for k in ['EM','Milstein']}; gdiff={k:[] for k in gerr}
gterm={k:[] for k in ['exact','EM','Milstein']}; batches={k:[] for k in gerr}
start=time.perf_counter()
for b in range(M_GBM//BATCH):
    fine=rng.normal(size=(ng[-1],BATCH))*np.sqrt(hg[-1]); exact=gbm(fine,hg[-1],'exact')
    errors={k:[] for k in gerr}; diffs={k:[] for k in gerr}
    for n,h in zip(ng,hg):
        dw=fine.reshape(n,ng[-1]//n,BATCH).sum(axis=1)
        for method in gerr:
            s=gbm(dw,h,method); errors[method].append(abs(s-exact)); diffs[method].append(s-exact)
            if n==32: gterm[method].append(s)
    gterm['exact'].append(exact)
    for method in gerr:
```

```
        gerr[method].append(np.array(errors[method])); gdiff[method].append(np.array(diffs[method]))
        batches[method].append(np.array(errors[method]).mean(axis=1))
G={}; analytic_mean=100*np.exp(.2); analytic_var=10000*np.exp(.4)*np.expm1(.09)
for method in gerr:
    gerr[method]=np.concatenate(gerr[method],axis=1); gdiff[method]=np.concatenate(gdiff[method],axis=1)
    G[method]={'strong':[summary(v) for v in gerr[method]],'weak_signed':[summary(v) for v in gdiff[method]],
               'slope':slope_ci(np.array(batches[method])[:,2:],hg[2:])}
for k in gterm: gterm[k]=np.concatenate(gterm[k])
bias_exact=100*np.exp(ng*np.log1p(.2/ng))-analytic_mean
G.update({'n':ng.tolist(),'analytic_mean':analytic_mean,'analytic_variance':analytic_var,
          'exact_mean_check':summary(gterm['exact']), 'exact_sample_variance':float(gterm['exact'].var(ddof=1)),
          'analytic_bias':bias_exact.tolist(),'elapsed_seconds':time.perf_counter()-start})
G['variance_check']=summary((gterm['exact']-analytic_mean)**2)
for method in ['EM','Milstein']:
    k=7 if method=='EM' else 9
    wb=-gdiff[method].reshape(9,M_GBM//BATCH,BATCH).mean(axis=2).T[:,:k]
    G[method]['weak_slope']=slope_ci(wb,hg[:k])
G['analytic_weak_slope']=float(np.polyfit(np.log(hg),np.log(abs(bias_exact)),1)[0])
for method in ['EM','Milstein']:
    print(method,'strong slope, bootstrap 95% CI:',G[method]['slope'])
    print('Weak slope and bootstrap CI:',G[method]['weak_slope'])
    print(' n     strong L1 (SE)        signed weak bias (SE)      analytic bias')
    for j,n in enumerate(ng):
        a=G[method]['strong'][j]; d=G[method]['weak_signed'][j]
        print(f'{n:4}  {a["mean"]:.6f} ({a["se"]:.6f})  {d["mean"]:+.6f} ({d["se"]:.6f})  {bias_exact[j]:+.6f}')
print('Exact-law moments:', {k:G[k] for k in ['analytic_mean','analytic_variance','exact_mean_check','exact_sample_variance']})
fig,ax=plt.subplots(1,2,figsize=(10,4))
for method in ['EM','Milstein']:
    ss=G[method]['strong']; ww=G[method]['weak_signed']
    ax[0].errorbar(hg,[v['mean'] for v in ss],yerr=[1.96*v['se'] for v in ss],marker='o',label=method)
    ax[1].errorbar(hg,-np.array([v['mean'] for v in ww]),yerr=[1.96*v['se'] for v in ww],marker='o',label=method)
ax[1].plot(hg,-bias_exact,'k--',label='analytic bias magnitude')
for a in ax: a.set_xscale('log'); a.set_yscale('log'); a.set_xlabel('step h'); a.legend()
ax[1].set_yscale('symlog',linthresh=.003); ax[1].axhline(0,c='gray',ls=':')
ax[0].set_ylabel('mean absolute terminal error'); ax[1].set_ylabel('negative signed mean bias'); savefig('gbm_convergence')
q=np.array([.01,.05,.5,.95,.99]); qe=100*np.exp(.155+.3*ndtri(q))
bins=np.linspace(0,400,81)
G['histogram']={'centers':(.5*(bins[1:]+bins[:-1])).tolist(),**{k:(np.histogram(v,bins)[0]/(len(v)*5)).tolist() for k,v in gterm.items()}}
G['quantiles']={'p':q.tolist(),'analytic':qe.tolist(),**{k:np.quantile(v,q).tolist() for k,v in gterm.items()}}
fig,ax=plt.subplots(1,2,figsize=(10,4)); xx=np.linspace(25,300,500)
density=np.exp(-.5*((np.log(xx/100)-.155)/.3)**2)/(xx*.3*np.sqrt(2*np.pi))
for k in ['exact','EM','Milstein']: ax[0].hist(gterm[k],bins=80,density=True,histtype='step',label=k)
ax[0].plot(xx,density,'k--',label='lognormal density'); ax[0].set_xlim(25,300); ax[0].legend(); ax[0].set_xlabel('terminal price'); ax[0].set_ylabel('density')
for k in ['exact','EM','Milstein']: ax[1].plot(qe,G['quantiles'][k],'o-',label=k)
ax[1].plot(qe,qe,'k--'); ax[1].set_xlabel('analytic quantile'); ax[1].set_ylabel('sample quantile'); ax[1].legend(); savefig('gbm_distribution')
RESULTS['gbm']=G
# %% Non-affine model: all reference grids use the same Brownian path
ns=np.array([8,16,32,64,128,256,1024,2048,4096]); hs=1/ns
rng=np.random.default_rng(np.random.SeedSequence([SEED,2])); st=[]; yt=[]; bt=[]; moment=np.zeros(5); ninc=0
start=time.perf_counter()
for b in range(M_SV//BATCH):
    z1=rng.normal(size=(4096,BATCH)); z2=rng.normal(size=z1.shape)
    d1=z1/64; d2=(-.7*z1+np.sqrt(.51)*z2)/64
    moment += [d1.sum(),d2.sum(),(d1*d1).sum(),(d2*d2).sum(),(d1*d2).sum()]; ninc+=d1.size
    ss=[]; yy=[]; bb=[]
    for n,h in zip(ns,hs):
        a=d1.reshape(n,4096//n,BATCH).sum(axis=1); c=d2.reshape(n,4096//n,BATCH).sum(axis=1)
        s,y=sv(a,c,h); ss.append(s); yy.append(y)
        if n<=256: bb.append(sv(a,c,h,balanced=True)[0])
    st.append(ss); yt.append(yy); bt.append(bb)
st=np.concatenate(st,axis=1); yt=np.concatenate(yt,axis=1); bt=np.concatenate(bt,axis=1)
V={'n':ns.tolist(),'elapsed_seconds':time.perf_counter()-start,'reference_comparisons':{}}
means=moment[:2]/ninc; variances=moment[2:4]/ninc-means**2
V['brownian']={'count':ninc,'means':means.tolist(),'variances':variances.tolist(),
               'correlation':float((moment[4]/ninc-means.prod())/np.sqrt(variances.prod()))}
for ref in [1024,2048,4096]:
```

```
    j=int(np.where(ns==ref)[0][0]); err=abs(st[:6]-st[j]); payoff=np.maximum(st[:6]-100,0)-np.maximum(st[j]-100,0)
    batch=err.reshape(6,M_SV//BATCH,BATCH).mean(axis=2).T
    V['reference_comparisons'][str(ref)]={'strong':[summary(a) for a in err], 'payoff_bias':[summary(a) for a in payoff], 'slope':slope_ci(batch[:,:5],hs[:5])}
V['refinement']={}
for a,b in [(1024,2048),(2048,4096),(1024,4096)]:
    i=int(np.where(ns==a)[0][0]); j=int(np.where(ns==b)[0][0])
    V['refinement'][f'{a}-{b}']={'strong':summary(abs(st[i]-st[j])),
      'payoff':summary(np.maximum(st[i]-100,0)-np.maximum(st[j]-100,0))}
V['terminal']={'mean':summary(st[-1]),'payoff':summary(np.maximum(st[-1]-100,0)),
 'min_price':float(st.min()),'max_price':float(st.max()),'min_y':float(yt.min()),'max_y':float(yt.max()),
 'analytic_mean':100*np.exp(.05)}
# An exact discrete martingale identity validates E[S_n] for log EM.
print('Brownian validation:',V['brownian']); print('Reference terminal diagnostics:',V['terminal'])
for ref,r in V['reference_comparisons'].items():
 print('Reference',ref,'empirical slope and CI',r['slope'])
 for n,a,p in zip(ns[:6],r['strong'],r['payoff_bias']): print(f'n={n:3} L1={a["mean"]:.6f} +/- {1.96*a["se"]:.6f}; payoff bias={p["mean"]:+.6f} +/- {1.96*p["se"]:.6f}')
print('Reference refinement:',V['refinement'])
fig,ax=plt.subplots(1,2,figsize=(10,4))
for ref,r in V['reference_comparisons'].items():
 ax[0].errorbar(hs[:6],[v['mean'] for v in r['strong']],yerr=[1.96*v['se'] for v in r['strong']],marker='o',label='ref 1/'+ref)
 ax[1].errorbar(hs[:6],[v['mean'] for v in r['payoff_bias']],yerr=[1.96*v['se'] for v in r['payoff_bias']],marker='o',label='ref 1/'+ref)
ax[0].set_yscale('log'); ax[0].set_ylabel('mean absolute terminal difference'); ax[1].set_ylabel('signed call-payoff difference'); ax[1].axhline(0,c='k',ls='--')
for a in ax: a.set_xscale('log'); a.set_xlabel('step h'); a.legend()
savefig('nonaffine_convergence'); RESULTS['nonaffine']=V
# %% Cost at a common GBM pathwise accuracy target
# Time only integration on pre-generated increments, 7 repeats after warm-up.
target=.5; cost={}; rng=np.random.default_rng(np.random.SeedSequence([SEED,3]))
for method in ['EM','Milstein']:
    acceptable=[j for j,a in enumerate(G[method]['strong']) if a['hi']<=target]
    assert acceptable, 'Extend grid for matched target'
    j=acceptable[0]; n=int(ng[j]); dw=rng.normal(size=(n,8192))/np.sqrt(n)
    gbm(dw,1/n,method); ts=[]
    for _ in range(7):
        tic=time.perf_counter(); gbm(dw,1/n,method); ts.append(time.perf_counter()-tic)
    cost[method]={'n':n,'error':G[method]['strong'][j], 'seconds_median':float(np.median(ts)), 'seconds_range':[min(ts),max(ts)], 'path_updates':n*8192}
dw=rng.normal(size=(1,8192)); gbm(dw,1,'exact'); ts=[]
for _ in range(7):
 tic=time.perf_counter(); gbm(dw,1,'exact'); ts.append(time.perf_counter()-tic)
cost['exact']={'n':1,'seconds_median':float(np.median(ts)),'path_updates':8192}
RESULTS['cost']={'target':target,'paths':8192,'methods':cost}; print('Matched-accuracy cost:',RESULTS['cost'])
fig,ax=plt.subplots(figsize=(6,4)); labels=list(cost); ax.bar(labels,[cost[k]['seconds_median']*1000 for k in labels]); ax.set_ylabel('median integration time (ms), 8192 paths'); ax.set_title('GBM terminal L1 target <= 0.5'); savefig('cost')
# %% Positivity: analytic one-step probabilities and independent stress samples
rng=np.random.default_rng(np.random.SeedSequence([SEED,4])); positivity=[]
for label,mu,sigma in [('baseline',.2,.3),('stress',.05,1.5)]:
 for h in [1.,.5,.25,.125]:
    z=rng.normal(size=200000); em=1+mu*h+sigma*np.sqrt(h)*z
    mi=em+.5*sigma*sigma*h*(z*z-1)
    minimum=.5+(mu-.5*sigma*sigma)*h
    p_em=float(ndtr(-(1+mu*h)/(sigma*np.sqrt(h))))
    if minimum<0:
      root=np.sqrt(-2*minimum/(sigma*sigma*h)); center=-1/(sigma*np.sqrt(h))
      p_mi=float(ndtr(center+root)-ndtr(center-root))
    else:p_mi=0.
    row={'case':label,'h':h,'EM_probability':p_em,'EM_count':int((em<0).sum()),
         'Milstein_probability':p_mi,'Milstein_count':int((mi<0).sum()),'samples':len(z)}
    positivity.append(row); print(row)
RESULTS['positivity']=positivity
# %% Derived extension: moment-balanced factor update and payoff control variate
E={'strong':[summary(abs(v-st[-1])) for v in bt],
   'payoff_bias':[summary(np.maximum(v-100,0)-np.maximum(st[-1]-100,0)) for v in bt]}
batch=abs(bt-st[-1]).reshape(6,M_SV//BATCH,BATCH).mean(axis=2).T
E['slope']=slope_ci(batch[:,:5],hs[:5]); print('Moment-balanced method:',E)
# Reference payoff CV: beta uses a disjoint pilot, evaluation uses remaining paths.
pilot=4096; stock=st[-1]; call=np.maximum(stock-100,0)
beta=float(np.cov(stock[:pilot],call[:pilot],ddof=1)[0,1]/stock[:pilot].var(ddof=1))
```

```
raw=call[pilot:]; controlled=raw-beta*(stock[pilot:]-100*np.exp(.05))
E['control_variate']={'pilot':pilot,'evaluation':len(raw),'beta':beta,'raw':summary(raw),
 'controlled':summary(controlled),'variance_reduction':float(raw.var(ddof=1)/controlled.var(ddof=1))}
# The model and log-EM both have exactly known first price moment.
print('Independent-pilot payoff control variate:',E['control_variate'])
# Stability / stationary second moment stress experiment (T=40).
rng=np.random.default_rng(np.random.SeedSequence([SEED,5])); stationary=[]
vstar=.36*(1+.04)/(4-.36)
for h in [1.,.5,.25,.125]:
 n=round(40/h); dw=rng.normal(size=(n,32768))*np.sqrt(h)
 for balanced in [False,True]:
  y=np.full(32768,-.2); a=np.exp(-2*h); c=.6*np.sqrt(-np.expm1(-4*h)/4)
  for w in dw:
   y=(-.2+a*(y+.2)+c*np.sqrt(1+y*y)*w/np.sqrt(h)) if balanced else y+2*(-.2-y)*h+.6*np.sqrt(1+y*y)*w
  second=summary((y+.2)**2)
  A=a*a+.36*(-np.expm1(-4*h))/4 if balanced else (1-2*h)**2+.36*h
  theoretical=vstar if balanced else (.36*(1+.04)/(4-.36-4*h) if A<1 else None)
  stationary.append({'h':h,'method':'balanced' if balanced else 'EM','second_about_theta':second,
   'A':float(A),'stationary_variance':theoretical,'finite_T_expected_second':float(.36*(1+.04)*(1-a*a)/4*(1-A
    **n)/(1-A)) if balanced else float(.36*(1+.04)*h*(1-A**n)/(1-A))})
E['stationary']=stationary; print('Factor stability stress:',stationary)
# Match a common empirical reference-error ceiling for the two SV schemes.
cs={}; rng=np.random.default_rng(np.random.SeedSequence([SEED,6]))
for name,errs,balanced in [('EM',V['reference_comparisons']['4096']['strong'],False),('balanced',E['strong'],
    True)]:
 j=next(j for j,e in enumerate(errs) if e['hi']<.5); n=int(ns[j]); h=1/n
 w1=rng.normal(size=(n,8192))*np.sqrt(h); w2=-.7*w1+np.sqrt(.51*h)*rng.normal(size=w1.shape)
 sv(w1,w2,h,balanced=balanced); ts=[]
 for _ in range(7):
  tic=time.perf_counter(); sv(w1,w2,h,balanced=balanced); ts.append(time.perf_counter()-tic)
 cs[name]={'n':n,'error':errs[j],'seconds_median':float(np.median(ts)),'path_updates':n*8192}
E['cost']={'target':.5,'paths':8192,'methods':cs}; print('SV matched reference-accuracy cost:',E['cost'])
fig,ax=plt.subplots(1,2,figsize=(10,4))
for name,vals in [('EM',V['reference_comparisons']['4096']['strong']),('balanced',E['strong'])]:
 ax[0].errorbar(hs[:6],[v['mean'] for v in vals],yerr=[1.96*v['se'] for v in vals],marker='o',label=name)
ax[0].set_xscale('log');ax[0].set_yscale('log');ax[0].set_xlabel('step h');ax[0].set_ylabel('mean absolute
    terminal difference');ax[0].legend()
for name in ['EM','balanced']:
 rows=[r for r in stationary if r['method']==name and r['h']<1]
 ax[1].plot([r['h'] for r in rows],[r['second_about_theta']['mean'] for r in rows],'o-',label=name)
ax[1].axhline(vstar,c='k',ls='--',label='exact stationary variance');ax[1].set_xlabel('step h');ax[1].
    set_ylabel('E[(Y(40)-theta)^2]');ax[1].legend();savefig('extension')
RESULTS['extension']=E
# %% Transient animation: prices, volatility and the factor on shared paths
from matplotlib.animation import FuncAnimation, PillowWriter
rng=np.random.default_rng(np.random.SeedSequence([SEED,7])); n=256; h=1/n
w1=rng.normal(size=(n,6))*np.sqrt(h);w2=-.7*w1+np.sqrt(.51*h)*rng.normal(size=w1.shape)
xx=np.full((n+1,6),np.log(100.)); yy=np.zeros((n+1,6))
for i in range(n):
 g=.1+.4*expit(yy[i]);xx[i+1]=xx[i]+(.05-.5*g*g)*h+g*w1[i]
 yy[i+1]=yy[i]+2*(-.2-yy[i])*h+.6*np.sqrt(1+yy[i]**2)*w2[i]
prices=np.exp(xx);vol=.1+.4*expit(yy); tt=np.arange(n+1)*h
fig,axes=plt.subplots(2,1,figsize=(7,5),sharex=True);lines=[]
for ax,data,label in zip(axes,[prices,vol],['price S','volatility g(Y)']):
 for j in range(6): lines.append((ax.plot([],[],lw=1)[0],data[:,j]))
 ax.set_xlim(0,1);ax.set_ylim(data.min()*.95,data.max()*1.05);ax.set_ylabel(label)
axes[-1].set_xlabel('time (years)')
def animate(k):
 for line,data in lines: line.set_data(tt[:k+1],data[:k+1])
 return [item[0] for item in lines]
ani=FuncAnimation(fig,animate,frames=range(0,n+1,4),interval=70,blit=True)
ani.save(OUT/'paths.gif',writer=PillowWriter(fps=14));plt.close(fig)
if 'show_animation' in globals(): show_animation(OUT/'paths.gif')
print('Animation generated: outputs/paths.gif (six simulated paths, fixed seed).')
# %% Independent checks and export
checks={}
rng=np.random.default_rng(44); d=rng.normal(size=(16,100))/4
checks['exact_transition_composition']=bool(np.allclose(100*np.prod(np.exp(.155/16+.3*d),axis=0),gbm(d,1/16,'
    exact'),rtol=1e-13))
checks['zero_noise_EM_equals_Milstein']=bool(np.array_equal(gbm(d,1/16,'EM',sigma=0),gbm(d,1/16,'Milstein',
    sigma=0)))
x=100.; w=.12; h=.1
checks['scalar_Milstein_formula']=bool(np.isclose(gbm(np.array([[w]]),h,'Milstein')[0],x*(1+.2*h+.3*w
    +.5*.09*(w*w-h))))
# xi=0, theta=Y0=0 freezes Y; log EM must equal exact GBM with sigma=g(0)=.3.
```

```
s,y=sv(d,d,1/16,mu=.2,theta=0,xi=0)
checks['nonaffine_frozen_factor_reduces_to_GBM']=bool(np.allclose(s,gbm(d,1/16,'exact'),rtol=1e-13))
checks['Brownian_variance']=bool(np.all(abs(np.array(V['brownian']['variances'])*4096-1)<.002))
checks['Brownian_correlation']=bool(abs(V['brownian']['correlation']+.7)<.002)
checks['GBM_moment_within_4SE']=bool(abs(G['exact_mean_check']['mean']-analytic_mean)<4*G['exact_mean_check']['se'])
checks['SV_martingale_within_4SE']=bool(abs(V['terminal']['mean']['mean']-100*np.exp(.05))<4*V['terminal']['mean']['se'])
checks['EM_order']=.45<G['EM']['slope'][0]<.6
checks['Milstein_order']=.9<G['Milstein']['slope'][0]<1.1
checks['balanced_frozen_factor']=bool(np.allclose(sv(d,d,1/16,mu=.2,theta=0,xi=0,balanced=True)[0],gbm(d,1/16,'exact'),rtol=1e-13))
checks['variance_identity']=bool(np.isclose((np.exp(-.4)**2+.36*(-np.expm1(-.8))/4)*vstar+.36*(-np.expm1(-.8))/4*(1+.04),vstar))
assert all(checks.values()), checks
RESULTS['checks']=checks
(OUT/'results.json').write_text(json.dumps(RESULTS,indent=2))
print('Validation checks:',json.dumps(checks,indent=2))
print('All outputs written to',OUT.resolve())

# Submission visualization checks: bootstrap whole independent batches.
from submission_figures import supplementary_figures
supplementary_figures(OUT, G, V, batches, gerr, st, hg, hs, M_SV, BATCH)

# Independent whole-experiment seeds required by the visualization guide.
from repeat_seeds import independent_seed_check
independent_seed_check(OUT)
```

Importable kernels used by the special-case tests

```
# %% Setup and reproducibility
from pathlib import Path
import sys, platform, time, json, math
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.special import expit, ndtr, ndtri
OUT = Path(__file__).resolve().parents[1] / 'outputs'; OUT.mkdir(exist_ok=True)
plt.rcParams.update({'font.size':10, 'axes.grid':True, 'grid.alpha':.25,
                     'figure.dpi':160, 'savefig.dpi':250, 'savefig.bbox':'tight'})
SEED=20260916
M_GBM=65536; M_SV=32768; BATCH=1024
RESULTS={'seed':SEED, 'python':sys.version.split()[0], 'numpy':np.__version__,
         'platform':platform.platform(), 'M_GBM':M_GBM,'M_SV':M_SV}
def summary(x):
    x=np.asarray(x); m=float(x.mean()); se=float(x.std(ddof=1)/np.sqrt(x.size))
    return {'mean':m,'se':se,'lo':m-1.96*se,'hi':m+1.96*se}
def slope_ci(batch_errors, hs, seed=17):
    # Resample independent path batches; retain dependence across levels.
    a=np.asarray(batch_errors); hs=np.asarray(hs)
    fit=lambda v: np.polyfit(np.log(hs),np.log(v),1)[0]
    rng=np.random.default_rng(seed)
    slopes=[fit(a[rng.integers(len(a),size=len(a))].mean(axis=0)) for _ in range(500)]
    return [float(fit(a.mean(axis=0))), *np.quantile(slopes,[.025,.975]).tolist()]
def savefig(name):
    plt.tight_layout(); plt.savefig(OUT/(name+'.png')); plt.savefig(OUT/(name+'.pdf'))
    show_figure(plt.gcf()) if 'show_figure' in globals() else None
    plt.close()
def gbm(dW,h,method,mu=.2,sigma=.3,s0=100.):
    # Rows are time, columns independent paths; no Python loop over paths.
    if method=='exact': return s0*np.exp((mu-.5*sigma*sigma)*dW.shape[0]*h+sigma*dW.sum(axis=0))
    f=1+mu*h+sigma*dW
    if method=='Milstein': f=f+.5*sigma*sigma*(dW*dW-h)
    return s0*np.prod(f,axis=0)
def sv(d1,d2,h,mu=.05,kappa=2.,theta=-.2,xi=.6, balanced=False):
    x=np.full(d1.shape[1],np.log(100.)); y=np.zeros_like(x)
    decay=np.exp(-kappa*h)
    noise=xi*np.sqrt(-np.expm1(-2*kappa*h)/(2*kappa*h))
    for a,b in zip(d1,d2):
        g=.1+.4*expit(y)
        x += (mu-.5*g*g)*h+g*a
```

```
        if balanced:
            y=theta+decay*(y-theta)+noise*np.sqrt(1+y*y)*b
        else:
            y += kappa*(theta-y)*h+xi*np.sqrt(1+y*y)*b
        assert np.all(np.isfinite(y)) and np.all(np.abs(x)<700)
    s=np.exp(x)
    assert np.all(np.isfinite(s)) and np.all(s>0) and np.all(np.isfinite(y))
    return s,y
```

Supplementary rate and cost figures

```
"""Additional evidence requested by the course visualization guide."""
import json
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt


def fit_band(ax, hs, batch_errors, theoretical_order, label, color, seed):
    """Resample independent batches, keeping the whole grid vector together."""
    x = np.log(np.asarray(hs))
    batch_errors = np.asarray(batch_errors)
    rng = np.random.default_rng(seed)
    grid = np.linspace(x.min(), x.max(), 100)
    coefs = np.array([
        np.polyfit(x, np.log(batch_errors[
            rng.integers(len(batch_errors), size=len(batch_errors))
        ].mean(axis=0)), 1) for _ in range(500)
    ])
    predicted = coefs[:, :1] * grid + coefs[:, 1:]
    lo, hi = np.quantile(predicted, [.025, .975], axis=0)
    central = np.polyfit(x, np.log(batch_errors.mean(axis=0)), 1)
    ax.fill_between(np.exp(grid), np.exp(lo), np.exp(hi), color=color,
                    alpha=.22, label=label + ' batch-bootstrap 95% band')
    ax.plot(np.exp(grid), np.exp(central[0]*grid+central[1]), color=color,
            ls='--', label=f'{label} fit p={central[0]:.4f}')
    theory = batch_errors.mean(axis=0)[-1] * (np.exp(grid)/hs[-1])**theoretical_order
    ax.plot(np.exp(grid), theory, color=color, ls=':',
            label=f'{label} reference order {theoretical_order:g}')
    return dict(slope=float(central[0]),
                slope_ci=np.quantile(coefs[:, 0], [.025, .975]).tolist(),
                h=np.exp(grid).tolist(), fit_lower=np.exp(lo).tolist(),
                fit_upper=np.exp(hi).tolist())


def supplementary_figures(out, gbm, sv, batches, gerr, st, hg, hs, m_sv, batch):
    fits = {}
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
    for method, color in [('EM', 'tab:blue'), ('Milstein', 'tab:orange')]:
        rows = gbm[method]['strong']
        axes[0].errorbar(hg, [v['mean'] for v in rows],
                         yerr=[1.96*v['se'] for v in rows], marker='o',
                         color=color, label=method + ' data')
        fits[method] = fit_band(axes[0], hg[2:], np.asarray(batches[method])[:, 2:],
                                .5 if method == 'EM' else 1., method, color, 17)
    ref = sv['reference_comparisons']['4096']
    axes[1].errorbar(hs[:6], [v['mean'] for v in ref['strong']],
                     yerr=[1.96*v['se'] for v in ref['strong']], marker='s',
                     color='tab:blue', label='EM vs 4096-step reference')
    e = abs(st[:6]-st[-1]).reshape(6, m_sv//batch, batch).mean(axis=2).T
    fits['nonaffine'] = fit_band(axes[1], hs[:5], e[:, :5], .5, 'EM', 'tab:blue', 17)
    for ax in axes:
        ax.set_xscale('log'); ax.set_yscale('log')
        ax.set_xlabel('step h [year]')
        ax.set_ylabel('terminal L1 price error [price units]')
        ax.legend(fontsize=7)
    axes[0].set_title('GBM: measured rates follow orders 1/2 and 1')
    axes[1].set_title('Non-affine EM: empirical half order, finite reference')
    fig.tight_layout()
    for ext in ['png', 'pdf']:
        fig.savefig(out / ('convergence_bands.'+ext), dpi=250, bbox_inches='tight')
    plt.close(fig)
```

```
    (out/'fit_bands.json').write_text(json.dumps(fits, indent=2))

    # Use the archived measurement quoted by the report, not a newly timed run.
    recorded = json.loads((Path(out).parent/'archived_outputs/results.json').read_text())
    fig, axes = plt.subplots(1, 2, figsize=(10, 4.2))
    for ax, cost, title in [
        (axes[0], recorded['cost'], 'GBM: Milstein needs less integration work'),
        (axes[1], recorded['extension']['cost'], 'Non-affine: no general speed advantage')
    ]:
        for method, color, marker in [('EM', 'tab:blue', 'o'),
                                      ('Milstein', 'tab:orange', 's'),
                                      ('balanced', 'tab:orange', 's')]:
            if method not in cost['methods']:
                continue
            v = cost['methods'][method]
            x = v['seconds_median']*1000
            ax.errorbar(x, v['error']['mean'], yerr=1.96*v['error']['se'],
                        color=color, marker=marker, ms=8, label=f'{method}, N={v["n"]}')
        ax.axhline(cost['target'], color='black', ls='--', label='common error ceiling 0.5')
        ax.set_xlabel('recorded median integration wall time [ms]')
        ax.set_ylabel('terminal L1 price discrepancy [price units]')
        ax.set_title(title); ax.legend(fontsize=8)
    fig.tight_layout()
    for ext in ['png', 'pdf']:
        fig.savefig(out / ('error_vs_cost.'+ext), dpi=250, bbox_inches='tight')
    plt.close(fig)
```

Independent whole-experiment seed repetitions

```
"""Independent whole-experiment seeds for the visualization guide.

These smaller supplementary runs do not replace the main convergence table.
Student-t bands quantify the mean fitted log-error across independent seeds.
They do not cover numerical-reference bias or model-fit error.
"""
import json
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import t
from kernels import gbm, sv


def independent_seed_check(out):
    grids = np.array([8, 16, 32, 64, 128, 256])
    finest, paths, batch = 4096, 4096, 512
    errors = {'EM': [], 'Milstein': [], 'nonaffine': []}
    seeds = [20261001, 20261002, 20261003, 20261004, 20261005]
    for seed in seeds:
        sums = {name: np.zeros(len(grids)) for name in errors}
        rng = np.random.default_rng(seed)
        for _ in range(paths // batch):
            d1 = rng.normal(size=(finest, batch)) / np.sqrt(finest)
            d2 = -.7*d1 + np.sqrt(1-.7**2)*rng.normal(size=d1.shape)/np.sqrt(finest)
            exact = gbm(d1, 1/finest, 'exact')
            ref = sv(d1, d2, 1/finest)[0]
            for j, n in enumerate(grids):
                w1 = d1.reshape(n, finest//n, batch).sum(axis=1)
                w2 = d2.reshape(n, finest//n, batch).sum(axis=1)
                for method in ['EM', 'Milstein']:
                    sums[method][j] += abs(gbm(w1, 1/n, method)-exact).sum()
                sums['nonaffine'][j] += abs(sv(w1, w2, 1/n)[0]-ref).sum()
        for name in errors:
            errors[name].append((sums[name]/paths).tolist())
    h = 1/grids
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
    fits = {}
    for name, ax, color, order, window in [
        ('EM', axes[0], 'tab:blue', .5, slice(1, None)),
        ('Milstein', axes[0], 'tab:orange', 1., slice(1, None)),
        ('nonaffine', axes[1], 'tab:blue', .5, slice(None, 5))
    ]:
```

```
        e = np.asarray(errors[name])
        coefficients = np.array([np.polyfit(np.log(h[window]), np.log(row[window]), 1)
                                 for row in e])
        lx = np.linspace(np.log(h[window]).min(), np.log(h[window]).max(), 100)
        lines = coefficients[:, :1]*lx + coefficients[:, 1:]
        center = lines.mean(axis=0)
        half = t.ppf(.975, len(seeds)-1)*lines.std(axis=0, ddof=1)/np.sqrt(len(seeds))
        for row in e:
            ax.loglog(h, row, color=color, alpha=.22, lw=.9)
        ax.loglog(h, e.mean(axis=0), 'o-', color=color, label=name+' mean error')
        ax.loglog(np.exp(lx), np.exp(center), '--', color=color, label=name+' fit')
        ax.fill_between(np.exp(lx), np.exp(center-half), np.exp(center+half),
                        color=color, alpha=.18, label='95% seed-mean fitted band')
        ax.loglog(np.exp(lx), e.mean(axis=0)[-1]*(np.exp(lx)/h[-1])**order,
                  ':', color=color, label=f'reference order {order:g}')
        p = coefficients[:, 0]
        hw = t.ppf(.975, len(seeds)-1)*p.std(ddof=1)/np.sqrt(len(seeds))
        fits[name] = {'seed_slopes': p.tolist(), 'mean_slope': float(p.mean()),
                      't95_interval_mean_slope': [float(p.mean()-hw), float(p.mean()+hw)]}
    for ax, title in zip(axes, ['GBM: independent seeds agree with theoretical orders',
                                'Non-affine EM: finite-reference half-order regime']):
        ax.set_title(title)
        ax.set_xlabel('step h [year]')
        ax.set_ylabel('terminal L1 price error [price units]')
        ax.legend(fontsize=7)
    fig.tight_layout()
    for ext in ['png', 'pdf']:
        fig.savefig(out/('independent_seed_bands.'+ext), dpi=250, bbox_inches='tight')
    plt.close(fig)
    record = {'seeds': seeds, 'paths_per_seed': paths, 'finest_grid': finest,
              'grids': grids.tolist(), 'errors': errors, 'fits': fits,
              'interpretation': 'approximate Student-t band for mean fitted log-error; no reference-bias
     coverage'}
    (out/'independent_seeds.json').write_text(json.dumps(record, indent=2))
    print('Independent seed check:', json.dumps(fits, indent=2))


if __name__ == '__main__':
    independent_seed_check(Path(__file__).resolve().parents[1]/'outputs')
```

Independent analytic checks

```
"""Independent analytic special-case checks; no full Monte Carlo run required."""
import unittest
import numpy as np
from kernels import gbm, sv


class AnalyticChecks(unittest.TestCase):
    def test_exact_transition_composition(self):
        dw = np.array([[.2, -.1], [-.3, .4], [.1, -.2]])
        h = .1
        oracle = 100*np.exp((.2-.09/2)*3*h+.3*dw.sum(axis=0))
        np.testing.assert_allclose(gbm(dw, h, 'exact'), oracle, rtol=1e-13)

    def test_zero_diffusion_deterministic_recurrence(self):
        dw = np.array([[.2], [-.3], [.1]])
        for method in ['EM', 'Milstein']:
            np.testing.assert_allclose(gbm(dw, .1, method, sigma=0),
                                       [100*(1+.2*.1)**3], rtol=1e-13)

    def test_one_step_milstein_independent_formula(self):
        h, w = .25, -.2
        oracle = 100*(1+.2*h+.3*w+.045*(w*w-h))
        np.testing.assert_allclose(gbm(np.array([[w]]), h, 'Milstein'), [oracle])

    def test_frozen_factor_exact_gbm(self):
        dw = np.array([[.2, -.1], [-.3, .4], [.1, -.2]])
        for balanced in [False, True]:
            price, factor = sv(dw, dw, .1, mu=.2, theta=0, xi=0,
                               balanced=balanced)
            oracle = 100*np.exp(.155*.3+.3*dw.sum(axis=0))
```

```
            np.testing.assert_allclose(price, oracle, rtol=1e-13)
            np.testing.assert_equal(factor, np.zeros(2))

    def test_no_shocks_log_scheme(self):
        dw = np.zeros((10, 2))
        price, _ = sv(dw, dw, .1, mu=.05, theta=0, xi=0)
        np.testing.assert_allclose(price, 100*np.exp(.05-.3**2/2), rtol=1e-13)

    def test_em_negative_tail_and_milstein_stress(self):
        dw = np.array([[-1.5]])
        self.assertLess(gbm(dw, 1, 'EM', mu=.05, sigma=1.5)[0], 0)
        self.assertLess(gbm(np.array([[-1/1.5]]), 1, 'Milstein',
                            mu=.05, sigma=1.5)[0], 0)

    def test_milstein_minimum_condition(self):
        # Evaluate exactly at the quadratic minimizer; no random sampling.
        for h in [.125, .25, .5, 1.]:
            oracle = 100*(.5+(.05-.5*1.5**2)*h)
            actual = gbm(np.array([[-1/1.5]]), h, 'Milstein', mu=.05, sigma=1.5)
            np.testing.assert_allclose(actual, [oracle], atol=1e-12)

    def test_balanced_stationary_variance_identity(self):
        kappa, xi, theta = 2., .6, -.2
        v = xi**2*(1+theta**2)/(2*kappa-xi**2)
        for h in [.125, .25, .5, 1.]:
            a = np.exp(-kappa*h)
            r = xi**2*(1-a*a)/(2*kappa)
            self.assertAlmostEqual((a*a+r)*v+r*(1+theta**2), v, places=13)


if __name__ == '__main__':
    unittest.main(verbosity=2)
```